DATA SCIENCE

Data Engineering & Pipelines

Warehouses, lakehouses, Parquet and Iceberg, CDC ingestion, dbt, Airflow, streaming, data quality and the cost audit that earns you credibility at a customer.

data-engineering · elt · dbt · airflow · snowflake · bigquery · parquet · iceberg · delta-lake · cdc · kafka · data-modelling · data-quality · warehouse-cost · fde

Study Pack · Comprehensive reference & interview prep · ~116 min read · 25,437 words

CONTENTS

1. The Data Layer You Inherit
2. OLTP vs OLAP: Warehouses, Lakes and Lakehouses
3. File Formats and Physical Layout
4. Table Formats: Delta, Iceberg and Hudi
5. Ingestion Patterns
6. ELT vs ETL and Why ELT Won
7. Data Modelling
8. SQL for Data Engineers
9. dbt: The Transformation Layer
10. Orchestration
11. Streaming
12. Data Quality and Contracts
13. Pipeline Reliability
14. Governance, Lineage and Privacy
15. Performance and Cost
16. Feeding AI and ML
17. Working With a Customer's Data Team
18. Interview Questions
19. Checklist and Lab Plan

The Data Layer You Inherit

You will almost never build a data platform from scratch. You will be dropped into a customer who already has seventeen years of accumulated decisions — a mainframe extract that runs at 02:00, a Snowflake account nobody can fully explain, four hundred stored procedures, a Power BI dataset that one analyst refreshes manually, and a Kafka cluster that was installed for a project that got cancelled. Your first job is not to improve any of it. Your first job is to build an accurate map, because every estimate you give and every architecture you draw will be wrong if the map is wrong.

5	60-80%	2-6 wks	>50%
QUESTIONS BEFORE ANY DESIGN	OF PIPELINE WORK IS INGESTION AND QUALITY	TYPICAL WAIT FOR PRODUCTION DATA ACCESS	OF WAREHOUSE SPEND IS USUALLY AVOIDABLE

The first five questions

Ask these in the first meeting, write the answers down verbatim, and send them back as a written summary. Half the value of a Forward Deployed Engineer in week one is forcing a customer to state things they have never written down.

1. **Where does the data originate, and who owns the system of record?**
   Not "where is the data" — where is it *born*. If an order is created in SAP S/4HANA and copied into Snowflake nightly, SAP is the system of record and Snowflake is a derivative. Anyone who wants to change the data must change it in SAP. If two systems both claim to be the system of record for customers, you have found the project's central political problem on day one.

2. **What is the required freshness, stated as a number?**
   "Real-time" is not an answer. Push until you get "the sales dashboard must reflect orders placed up to 15 minutes ago" or "finance only needs the previous business day by 07:00 IST". The difference between 15 minutes and 24 hours is the difference between a streaming architecture and a cron job — roughly a 5-10x difference in cost and complexity.

3. **What is the volume, stated as rows per day and bytes per day?**
   Then ask for the largest table's row count and the daily growth of the largest table. Teams routinely describe themselves as "big data" at 40 GB total, which fits in a laptop's RAM, and routinely under-describe a 2 TB/day clickstream as "some web logs".

4. **What happens today when a pipeline fails?**

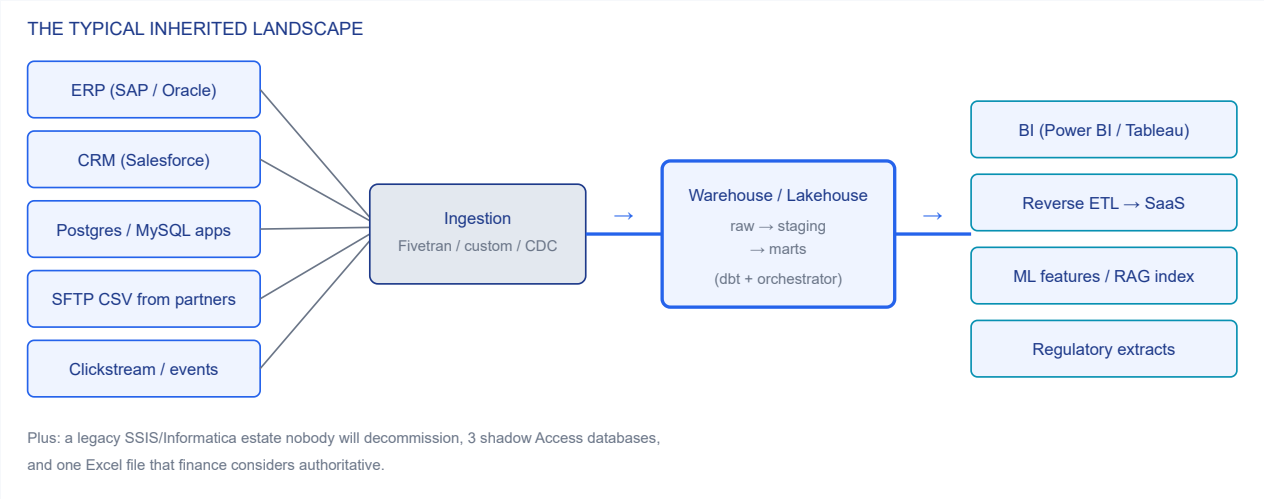
Who gets paged, what is the runbook, how often does it happen, and how long until anyone notices? If the answer is "an analyst notices the dashboard is stale and emails Rajesh", you have learned that there is no monitoring, no SLA, and no on-call, and that your deliverable must include those things or it will rot within a quarter.

5 **Who is allowed to see this data, and what is in it?**
PII, payment data, health data, employee data, data subject to data-residency rules. The answer determines whether you can take an extract to your own laptop, whether you can use a hosted LLM API on it, and whether the project needs a DPA before you write a line of code.

⚠ **The question behind the questions**
Everything above is really asking: **what decision does this data support, and what is the cost of that decision being wrong or late?** A pipeline that feeds a quarterly board slide and a pipeline that feeds a fraud block have the same SQL and completely different engineering budgets. Get the decision, then size the engineering.

What a realistic customer landscape looks like

Here is a composite that matches most mid-to-large enterprises you will walk into. Recognising the shape saves you weeks.



The landscape you inherit. The arrows on the left are where 70% of your effort goes.

Doing an honest inventory in week one

Read-only access plus half a day of SQL gives you more than three weeks of meetings. In Snowflake, these queries tell you what is actually used, which is rarely what the documentation says.

SQL

```sql
-- Which tables actually get queried? (Snowflake)
select
    lower(value:objectName::string)        as table_name,
    count(*)                               as query_count,
    count(distinct user_name)              as distinct_users,
    max(start_time)                        as last_queried
from snowflake.account_usage.access_history,
     lateral flatten(input => base_objects_accessed)
where query_start_time >= dateadd(day, -90, current_timestamp())
group by 1
order by query_count desc
limit 100;

-- Which tables are large but never read? (deletion candidates)
select t.table_catalog, t.table_schema, t.table_name,
       t.row_count,
       round(t.bytes / power(1024, 3), 2) as gb
from snowflake.account_usage.tables t
where t.deleted is null
  and t.bytes > 10 * power(1024, 3)
order by t.bytes desc
limit 50;

-- Where is the money going?
select warehouse_name,
       sum(credits_used)            as credits,
       sum(credits_used) * 3.0      as approx_usd   -- check your contract rate
from snowflake.account_usage.warehouse_metering_history
where start_time >= dateadd(day, -30, current_timestamp())
group by 1
order by credits desc;
```

The equivalent in BigQuery uses `INFORMATION_SCHEMA.JOBS`:

```sql
```

```sql
select
    user_email,
    count(*)                                            as jobs,
    round(sum(total_bytes_billed) / power(1024, 4), 2)    as tb_billed,
    round(sum(total_bytes_billed) / power(1024, 4) * 6.25, 2) as approx_usd
from `region-us`.INFORMATION_SCHEMA.JOBS_BY_PROJECT
where creation_time ≥ timestamp_sub(current_timestamp(), interval 30 day)
  and job_type = 'QUERY'
group by 1
order by tb_billed desc
limit 25;
```

✦ **The slide that wins you credibility**

Turn those three queries into one page: "Top 10 tables by usage", "Top 10 tables by cost that nobody reads", "Top 5 users by spend". Present it in week two. You will have told the customer something true about their own platform that they did not know, and you will have earned the right to propose changes.

What "done" means here

- [ ] A one-page diagram of sources, landing zone, transformation, consumers — reviewed by the customer's lead engineer and corrected.
- [ ] A table of the top 20 datasets with owner, freshness SLA, volume, PII flag, downstream consumers.
- [ ] A written list of every credential/access you need, with who approves each, and the current status.
- [ ] The three cost/usage queries above, scheduled to run weekly into a dashboard.

OLTP vs OLAP: Warehouses, Lakes and Lakehouses

Every argument about data platforms reduces to one physical fact: transactional systems are optimised to read and write a single row quickly, and analytical systems are optimised to read one column across a billion rows quickly. Row storage and column storage are different enough that no single engine is excellent at both, which is why your customer has at least two, and why the copying between them is your job.

The physical difference

Consider a table of 100 million orders with 40 columns. A query asks for `sum(amount) group by region`, touching two columns.

	Row store (Postgres, MySQL, Oracle)	Column store (Snowflake, BigQuery, Parquet)
Physical layout	All 40 columns of row 1, then all 40 of row 2…	All values of `amount` together, then all of `region` …
Bytes read for that query	~All of it. ~20 GB.	Two columns. ~400 MB, less after compression.
Compression ratio	2-3x (mixed types adjacent)	5-20x (same type and often similar values adjacent)
Single-row lookup by PK	Microseconds via B-tree index	Hundreds of milliseconds to seconds — must scan/prune
Single-row UPDATE	In place, cheap	Rewrite a whole file or micro-partition
Concurrency model	Thousands of short transactions/sec	Tens of long-running scans
Correct use	Serving the application	Answering questions about history

✕ **The number one anti-pattern you will be asked to build**
"Can we just point the dashboard at the production Postgres replica?" For a startup with 2 GB of data, yes, and you should say yes. At scale it means analysts' six-way joins compete for buffer cache with the checkout flow. The standard progression is: read replica → nightly dump into a warehouse → CDC into a warehouse. Know where the customer is on that line before you propose Kafka.

Warehouse, lake, lakehouse — the actual distinctions

The words are abused. Here is what they mean operationally.

DATA WAREHOUSE

Storage and compute are managed together by one vendor, data is loaded into proprietary (or vendor-optimised) storage, and SQL is the only interface. Strong consistency, strong governance, excellent performance, poor fit for images/audio/arbitrary files, and you pay the vendor for storage. Snowflake, BigQuery, Redshift, Synapse.

DATA LAKE

Files in object storage (S3, ADLS, GCS) in open formats, read by whatever engine you like. Cheap ($20-25/TB/month), infinitely flexible, handles any file type. Historically no transactions, no schema enforcement, no indexes — which is how lakes became swamps full of 4 KB JSON files nobody can query.

LAKEHOUSE

A lake plus a table format (Delta/Iceberg/Hudi) that adds ACID transactions, schema enforcement, time travel and statistics on top of the files. You get warehouse semantics on open storage you own, queryable by multiple engines. Databricks, Snowflake-on-Iceberg, Fabric, Athena/Trino + Iceberg.

The honest summary in 2025-2026 is that the categories have converged. Snowflake reads and writes Iceberg; Databricks SQL is a competent warehouse; BigQuery reads external Iceberg tables; Fabric puts Delta under everything. The question is no longer "warehouse or lake" but **"who owns the storage, and can a second engine read it without a migration?"**.

A practical comparison of the big five

Platform	Compute model	Pricing shape	Where it shines	Where it hurts
Snowflake	Virtual warehouses (T-shirt sizes), per-second billing after 60s minimum, auto-suspend	Credits/hour by warehouse size, + storage	Zero tuning, excellent concurrency isolation, zero-copy cloning, time travel, Snowpark	Costs creep silently; idle warehouses and over-sized warehouses are the classic leak
BigQuery	Serverless slots, no cluster to size	On-demand $/TB scanned, or slot reservations	Zero ops, massive scans, great for bursty/irregular workloads, BI Engine	A single `select *` on a 5 TB table costs real money; cost is query-shape sensitive
Databricks	Spark clusters + serverless SQL warehouses	DBUs + underlying cloud VMs	Unified ML/streaming/SQL, Delta + Unity Catalog, best for heavy Python/Spark and ML	More knobs than a warehouse; Spark expertise required; cluster startup latency
Redshift	Provisioned RA3 nodes or Serverless RPUs	Node-hours or RPU-hours + managed storage	Deep AWS integration, predictable fixed cost, Spectrum over S3	Manual tuning (dist/sort keys) still matters; concurrency scaling costs extra
Microsoft Fabric	Capacity units (F-SKUs) shared across all workloads	One capacity, all services	Power BI shops; OneLake with Delta under everything; simple commercials	Noisy-neighbour throttling across workloads on one capacity; maturing fast but young

> **Choosing for a customer**
> Nine times out of ten you should not choose. The customer has a cloud, a Microsoft or AWS enterprise agreement, and a team that knows one SQL dialect. Pick the thing that is already paid for and already understood. The exceptions worth fighting for: heavy ML/Python → Databricks; wildly bursty analytics with no ops team → BigQuery; strict workload isolation across many business units → Snowflake.

Rough cost intuition

Treat these as orders of magnitude and verify against current pricing pages — rates change and enterprise discounts are large.

Item	Rough rate	What it means in practice
Object storage	$20-25 / TB / month	Storing 100 TB of history is ~$2.5k/month — storage is almost never the problem
Snowflake credit	~$2-4 depending on edition	An X-Small warehouse left on 24/7 ≈ 1 credit/hr ≈ $2-3k/month for nothing
BigQuery on-demand	~$5-6.25 / TB scanned	An analyst running `select *` on a 2 TB table 20 times/day = ~$200-250/day
Databricks DBU	~$0.15-0.70 / DBU + VM cost	A 10-node job cluster for 1 hr/day is modest; an idle all-purpose cluster is not
Managed ingestion (Fivetran etc.)	Per monthly active row	Cheap until a high-churn table changes 50M rows/day, then suddenly the biggest line item

File Formats and Physical Layout

Format choice is the single highest-leverage, lowest-effort decision in a data platform. Converting a customer's CSV landing zone to properly sized, partitioned, Snappy-compressed Parquet routinely cuts scan cost by 10-30x and query time by a similar factor — for a day of work and no change to anyone's SQL. It is the easiest win you will ever present.

The format landscape

Format	Layout	Schema	Splittable	Typical use
CSV	Row, text	None (headers if lucky)	Yes (if no embedded newlines)	Interchange with humans and legacy systems. Never a storage format.
JSON / NDJSON	Row, text	Self-describing, unenforced	NDJSON yes, JSON array no	API payloads, event landing. Verbose: keys repeat on every row.
Avro	Row, binary	Embedded, strong evolution rules	Yes (sync markers)	Kafka messages, write-heavy ingestion, row-at-a-time replay
Parquet	Columnar, binary	Embedded + per-column stats	Yes (row groups)	The default for analytics. Universally supported.
ORC	Columnar, binary	Embedded + rich indexes	Yes (stripes)	Hive/Trino estates; slightly better compression, narrower ecosystem

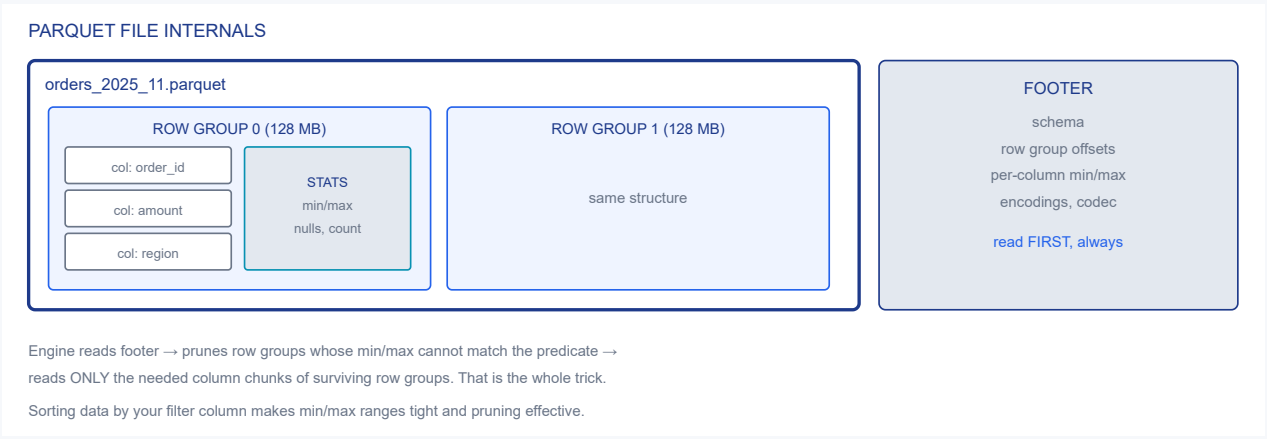
✓ **The rule**

Avro for the wire, Parquet for the shelf. Row-oriented binary for streaming ingestion where you write one record at a time and need strict schema evolution; columnar for anything that will be scanned analytically. If you remember one sentence from this section, remember that one.

Why Parquet is fast — the internals that matter

A Parquet file is a sequence of **row groups** (typically 128 MB), each containing one **column chunk** per column, each chunk split into **pages** (~1 MB). The footer holds the schema plus, crucially, **min/max**

statistics and null counts per column chunk.



Predicate pushdown works because of footer statistics. Unsorted data defeats it.

Encodings matter too. Parquet applies **dictionary encoding** to low-cardinality string columns (store each distinct value once, then store small integer codes), **run-length encoding** for repeated values, and **delta encoding** for sorted integers and timestamps. This is why sorting matters twice: it tightens statistics *and* makes RLE effective.

PYTHON

```python
import pyarrow as pa
import pyarrow.parquet as pq
import pandas as pd
import numpy as np

n = 5_000_000
df = pd.DataFrame({
    "order_id":   np.arange(n, dtype="int64"),
    "order_ts":   pd.Timestamp("2025-01-01") + pd.to_timedelta(np.random.randint(0, 86
    "region":     np.random.choice(["APAC", "EMEA", "NA", "LATAM"], n),   # low cardin
    "status":     np.random.choice(["NEW", "PAID", "SHIPPED", "CANCELLED"], n),
    "amount":     np.round(np.random.gamma(2.0, 120.0, n), 2),
    "customer_id":np.random.randint(1, 400_000, n),
})

df.to_csv("orders.csv", index=False)                                    # ~390 MB

table = pa.Table.from_pandas(df, preserve_index=False)
pq.write_table(table, "orders_snappy.parquet", compression="snappy")   # ~105 MB
pq.write_table(table, "orders_zstd.parquet",   compression="zstd")     # ~72 MB

# Sorting by the column you filter on tightens min/max stats dramatically
sorted_tbl = table.sort_by([("region", "ascending"), ("order_ts", "ascending")])
pq.write_table(sorted_tbl, "orders_sorted.parquet", compression="zstd",
               row_group_size=1_000_000,
               use_dictionary=["region", "status"],
               write_statistics=True)

# Inspect what you produced - do this on every customer dataset
meta = pq.ParquetFile("orders_sorted.parquet").metadata
print(f"rows={meta.num_rows:,} row_groups={meta.num_row_groups} size={meta.serialized_
rg = meta.row_group(0)
for i in range(rg.num_columns):
    c = rg.column(i)
    print(f"  {c.path_in_schema:12s} codec={c.compression:6s} "
          f"compressed={c.total_compressed_size:>10,} "
          f"uncompressed={c.total_uncompressed_size:>10,} "
          f"min={c.statistics.min if c.statistics else None}")
```

Compression codecs

Codec	Ratio	Compress speed	Decompress speed	Use when
Snappy	Baseline (~3-4x)	Very fast	Very fast	Default for hot, frequently scanned data
Zstd (level 3)	~1.3-1.5x better than Snappy	Fast	Fast	The modern default; usually strictly better than Snappy
Gzip	Similar to zstd	Slow	Slow	Legacy interop only
LZ4	Slightly worse than Snappy	Fastest	Fastest	CPU-bound streaming writes
Brotli	Best ratio	Very slow	Medium	Cold archival data read a few times a year

✦ **Default recommendation**

Zstd level 3 for almost everything on modern engines. If the customer's engine predates Zstd support (old Hive, old Spark), Snappy. Never Gzip for anything you query regularly — you pay the decompression tax on every scan forever.

Partitioning: the right way and the two wrong ways

Hive-style partitioning encodes partition values in the directory path, so an engine can skip entire directories without opening files:

TEXT

```
s3://lake/orders/
  event_date=2025-11-01/region=APAC/part-0000.parquet
  event_date=2025-11-01/region=EMEA/part-0000.parquet
  event_date=2025-11-02/region=APAC/part-0000.parquet
  ...
```

A query with `where event_date = '2025-11-02' and region = 'APAC'` opens exactly one directory. This is **partition pruning** and it is the cheapest performance you will ever buy.

WRONG WAY 1: TOO FINE

Partitioning by `customer_id` with 400,000 customers creates 400,000 directories each holding a 2 KB file. Every query now does hundreds of thousands of LIST and GET calls against object storage. The metadata operation dominates and the query takes minutes instead of seconds. Rule of thumb: aim for partitions of **at least 128 MB**, ideally 256 MB-1 GB.

WRONG WAY 2: TOO COARSE OR WRONG COLUMN

Partitioning by `year` only, when every query filters on a single day, means every query scans a year. Or partitioning by a column nobody filters on. Partition by what appears in `WHERE` clauses — overwhelmingly that is a date — and nothing else. Use clustering/sorting for the second dimension.

The small-file problem

This is the most common pathology in real customer lakes. A streaming job writes a file every 30 seconds, giving 2,880 files per day per partition. After a year you have a million files averaging 400 KB. Symptoms:

- [ ] Query planning takes longer than query execution — the engine spends minutes listing objects.
- [ ] S3 request charges appear as a significant line item (LIST/GET at ~$0.0004 per 1,000 requests adds up at tens of millions of requests).
- [ ] Spark jobs show thousands of tiny tasks, each with ~100 ms of overhead, so overhead exceeds work.
- [ ] Compression ratio is poor because each file has its own dictionary and no run-lengths.

The fix is **compaction**: periodically rewrite many small files into few large ones. Every table format gives you a command for this; in raw Parquet you write it yourself.

PYTHON

```python
# Generic compaction with DuckDB - works on any Parquet lake, no cluster needed.
# DuckDB will happily chew through tens of GB on a laptop or a small VM.
import duckdb

con = duckdb.connect()
con.execute("SET memory_limit='12GB'; SET threads=8;")
con.execute("INSTALL httpfs; LOAD httpfs;")

con.execute("""
COPY (
    SELECT * FROM read_parquet('s3://lake/raw/events/event_date=2025-11-01/*.parquet')
    ORDER BY tenant_id, event_ts          -- sort for tight min/max stats
)
TO 's3://lake/curated/events/event_date=2025-11-01/'
(FORMAT PARQUET,
 COMPRESSION ZSTD,
 ROW_GROUP_SIZE 1000000,
 FILE_SIZE_BYTES '256MB',                 -- target file size
 OVERWRITE_OR_IGNORE 1);
""")
```

SQL

```sql
-- Delta Lake equivalents
OPTIMIZE lake.events WHERE event_date = '2025-11-01';
OPTIMIZE lake.events ZORDER BY (tenant_id, customer_id);   -- multi-dim clustering
VACUUM  lake.events RETAIN 168 HOURS;                      -- drop old files (7 days)

-- Iceberg equivalents (Spark)
CALL catalog.system.rewrite_data_files(
  table => 'lake.events',
  strategy => 'sort',
  sort_order => 'tenant_id ASC, event_ts ASC',
  options => map('target-file-size-bytes', '268435456')
);
CALL catalog.system.expire_snapshots('lake.events', TIMESTAMP '2025-11-01 00:00:00');
CALL catalog.system.remove_orphan_files(table => 'lake.events');
```

⚠ **Compaction is not optional and it is not free**

Budget it as a scheduled job from day one, usually running an hour after the data lands. It costs compute. On a customer site, this is the thing that was never set up, and it is why their "simple" lake query takes eleven minutes. Finding it and fixing it is often your first visible win.

Sizing cheat sheet

Knob	Target	Why
File size	128 MB - 1 GB	Amortises open/LIST cost; matches HDFS/S3 read patterns
Row group size	128 MB (default) or ~1M rows	Unit of parallelism and of statistics granularity
Page size	1 MB (default)	Rarely worth changing
Partition size	> 128 MB, ideally 1 GB	Fewer, fatter partitions beat many thin ones
Partition count	< 10,000 per table if possible	Metadata operations become the bottleneck beyond that
Sort key	The column you filter on most, after the partition column	Tight min/max → row-group pruning

Table Formats: Delta, Iceberg and Hudi

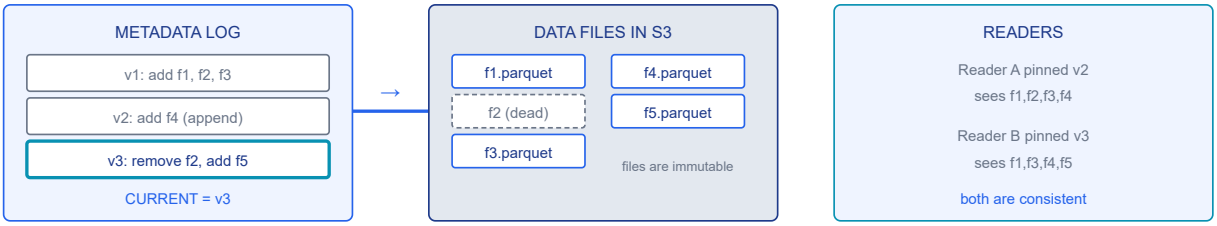
Object storage gives you durable, cheap, infinitely scalable files — and nothing else. No transactions, no schema enforcement, no way to know which files belong to "the table" right now. Table formats are a metadata layer on top of your Parquet files that restores database semantics. Understanding how they do it will make you the person in the room who can explain why a reader saw half an update.

The problem being solved

Imagine a naive lake table at `s3://lake/orders/` with 500 Parquet files. You want to delete customer 1234 for a GDPR erasure request. That means rewriting the 12 files that contain their rows. Between deleting the old files and writing the new ones, a reader doing `SELECT count(*)` sees an inconsistent table. If your job crashes halfway, the table is corrupt and there is no rollback. There is also no way to answer "what did this table look like last Tuesday".

Table formats fix this with one idea: **the table is not "whatever files are in this prefix" — the table is an explicit, versioned list of files.** Writers produce new files and then atomically publish a new version of the list. Readers pin a version at query start and see a consistent snapshot.

HOW A TABLE FORMAT MAKES OBJECT STORAGE TRANSACTIONAL



Atomicity comes from one atomic operation: publishing the new metadata pointer.
Delta: atomic PUT of _delta_log/000003.json (plus a commit service on S3).
Iceberg: atomic swap of the current-metadata pointer in the catalog (Glue/Nessie/REST/Hive).

Time travel is free: just read an older version of the file list. Nothing was deleted yet.

The whole mechanism: immutable data files plus an atomically-published, versioned file list.

Delta vs Iceberg vs Hudi

	Delta Lake	Apache Iceberg	Apache Hudi
Origin	Databricks	Netflix, now broadly governed	Uber
Metadata	JSON transaction log + periodic Parquet checkpoints in `_delta_log/`	Metadata file → manifest list → manifests → data files; pointer in a catalog	Timeline of commits + file groups/slices
Partition handling	Hive-style directories (plus Liquid Clustering)	**Hidden partitioning** — partition spec is metadata, not paths; can evolve	Hive-style
Partition evolution	Limited	Yes, without rewriting data	Limited
Schema evolution	Add/rename/reorder; type widening	Full, with stable column IDs (safe renames)	Add/widen
Upsert story	`MERGE INTO`, deletion vectors	`MERGE INTO`, copy-on-write and merge-on-read	Best-in-class record-level upserts; CoW and MoR
Engine support	Spark-first; broad via delta-rs, Trino, DuckDB, Fabric, Snowflake (read)	Broadest: Spark, Trino, Flink, Snowflake, BigQuery, Athena, Dremio, DuckDB, ClickHouse	Spark/Flink-first, narrower elsewhere
Pick it when	The customer is on Databricks or Fabric	You want engine independence or a multi-engine estate	Very high-frequency streaming upserts with incremental pulls

> **Where this is heading**
>
> Iceberg has become the de-facto interoperability format: Snowflake, BigQuery, Databricks (via UniForm), Athena and Trino all read it. If a customer asks "which format future-proofs us", Iceberg with a REST catalog is the defensible answer. If they are already deep in Databricks, Delta with UniForm enabled gives them both. Do not migrate a working Delta estate to Iceberg for ideology — migrate only if a second engine genuinely needs the data.

ACID on object storage, concretely

Here is a Delta table being created, updated, time-travelled and rolled back. Note that every operation is an atomic version bump.

PYTHON

```python
from pyspark.sql import SparkSession
from delta.tables import DeltaTable

spark = (SparkSession.builder
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
    .config("spark.sql.catalog.spark_catalog",
            "org.apache.spark.sql.delta.catalog.DeltaCatalog")
    .getOrCreate())

# v0: create
(spark.read.parquet("s3://lake/raw/orders/")
     .write.format("delta")
     .partitionBy("order_date")
     .mode("overwrite")
     .save("s3://lake/silver/orders"))

# v1: merge today's CDC batch (upsert + delete)
tgt = DeltaTable.forPath(spark, "s3://lake/silver/orders")
src = spark.read.json("s3://lake/cdc/orders/dt=2025-11-18/")

(tgt.alias("t")
    .merge(src.alias("s"), "t.order_id = s.order_id")
    .whenMatchedDelete(condition="s.op = 'D'")
    .whenMatchedUpdateAll(condition="s.op = 'U' AND s.updated_at > t.updated_at")
    .whenNotMatchedInsertAll(condition="s.op ≠ 'D'")
    .execute())

# Inspect history - this is the audit trail auditors actually accept
tgt.history().select("version", "timestamp", "operation",
                     "operationMetrics").show(truncate=False)

# Time travel
yesterday = spark.read.format("delta").option("timestampAsOf", "2025-11-17").load("s3:
v0        = spark.read.format("delta").option("versionAsOf", 0).load("s3://lake/silver/

# Did the merge do something insane? Roll back in one statement.
spark.sql("RESTORE TABLE delta.`s3://lake/silver/orders` TO VERSION AS OF 0")
```

SQL

```sql
-- The same ideas in Iceberg SQL
CREATE TABLE lake.silver.orders (
  order_id      bigint,
  customer_id   bigint,
  order_ts      timestamp,
  amount        decimal(12,2),
  status        string
) USING iceberg
PARTITIONED BY (days(order_ts))          -- hidden partitioning: no order_date column
TBLPROPERTIES (
  'write.target-file-size-bytes' = '268435456',
  'write.parquet.compression-codec' = 'zstd',
  'format-version' = '2'
);

MERGE INTO lake.silver.orders t
USING staged_cdc s ON t.order_id = s.order_id
WHEN MATCHED AND s.op = 'D' THEN DELETE
WHEN MATCHED AND s.op = 'U' THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;

-- Time travel and rollback
SELECT * FROM lake.silver.orders VERSION AS OF 3821;
SELECT * FROM lake.silver.orders FOR SYSTEM_TIME AS OF '2025-11-17 23:59:59';
CALL catalog.system.rollback_to_snapshot('lake.silver.orders', 3821);

-- Change partitioning WITHOUT rewriting history - Iceberg's party trick
ALTER TABLE lake.silver.orders REPLACE PARTITION FIELD days(order_ts) WITH hours(order
```

Copy-on-write vs merge-on-read

When you update one row inside a 256 MB file, you have two choices, and the choice is a write/read latency trade-off you will have to explain.

COPY-ON-WRITE (COW)

Rewrite the entire file with the change applied. **Writes are expensive** (amplification: 1 row changed → 256 MB rewritten) but **reads are fast** because files are clean Parquet. Default for Delta and Iceberg. Right for batch pipelines that update once an hour or once a day.

MERGE-ON-READ (MOR)

Write a small delete file or delta log alongside, and reconcile at query time. **Writes are cheap**, **reads pay a merge cost**, and you must compact regularly or read performance decays. Right for high-frequency streaming upserts. Hudi's speciality; available in Iceberg v2 and as Delta deletion vectors.

⚠ **The failure mode to watch for**

A customer enables MoR for write throughput, never schedules compaction, and six weeks later every read merges thousands of delete files. Query latency has quintupled and nobody knows why. If you enable MoR, you own the compaction schedule — put it in the DAG on the same day.

Retention, vacuum and the GDPR trap

Time travel works because old files are not deleted immediately. That is also a compliance hazard: if you "delete" a customer's row but the previous version is retained for 30 days, you have not actually erased it.

SQL

```sql
-- Delta: files older than the retention window become eligible for deletion
VACUUM delta.`s3://lake/silver/orders` RETAIN 168 HOURS;   -- 7 days

-- Iceberg: expire snapshots first, then clean orphans
CALL catalog.system.expire_snapshots(
  table => 'lake.silver.orders',
  older_than => TIMESTAMP '2025-11-11 00:00:00',
  retain_last => 5
);
CALL catalog.system.remove_orphan_files(table => 'lake.silver.orders');
```

- [ ] Pick a retention window deliberately: 7 days is a good default for operational rollback; 30+ days costs real storage and extends erasure latency.
- [ ] Write the window into the data-retention policy document and tell the privacy team the actual erasure SLA is "delete + vacuum", not "delete".
- [ ] Never run `VACUUM` with a retention shorter than your longest-running query, or you will delete files out from under a reader.

- [ ] Schedule `OPTIMIZE` + `VACUUM` (or `rewrite_data_files` + `expire_snapshots`) as explicit DAG tasks with monitoring.

Ingestion Patterns

Ingestion is where projects actually die. The transformation layer is SQL and it is pleasant; ingestion is credentials, firewalls, undocumented APIs, rate limits, partners who change a CSV header without telling anyone, and a mainframe that emits EBCDIC. Plan for ingestion to be 60-80% of the work and you will be approximately right.



The cardinal rule: land raw, untouched, forever

Whatever arrives from a source should be written to immutable storage exactly as received, with a timestamp and a batch identifier, *before* any parsing, casting or cleaning. This single discipline saves more projects than any other.

YOU CAN REPLAY

When you discover in March that your February timezone handling was wrong, you reprocess from raw. Without raw, the data is gone and you are writing an apology email.

YOU CAN PROVE PROVENANCE

"Our dashboard says 4,312 and yours says 4,318." You open the raw file for that day and settle it in ten minutes instead of three days.

YOU DECOUPLE SCHEDULES

Extraction and transformation become independent. A transformation bug no longer forces you to re-hit a rate-limited partner API.

TEXT

```
s3://lake/raw/{source}/{entity}/ingest_date=YYYY-MM-DD/run_id={uuid}/part-*.jsonl.gz
                                                              ^ never overwritten, never edi
```

Pattern 1: full-table batch extract

Pull the whole table every run. Crude, but correct, idempotent and trivial to reason about. Entirely appropriate below roughly 10-50 million rows.

PYTHON

```python
import os, uuid, datetime as dt
import sqlalchemy as sa
import pyarrow as pa, pyarrow.parquet as pq

def full_extract(table: str, out_root: str, chunk_rows: int = 500_000) -> dict:
    engine = sa.create_engine(os.environ["SOURCE_DSN"], pool_pre_ping=True)
    run_id = str(uuid.uuid4())
    day    = dt.date.today().isoformat()
    base   = f"{out_root}/{table}/ingest_date={day}/run_id={run_id}"
    rows, files = 0, 0

    with engine.connect().execution_options(stream_results=True, yield_per=chunk_rows)
        result = conn.execute(sa.text(f"select * from {table}"))
        batch  = []
        for row in result:
            batch.append(dict(row._mapping))
            if len(batch) ≥ chunk_rows:
                pq.write_table(pa.Table.from_pylist(batch),
                               f"{base}/part-{files:05d}.parquet", compression="zstd")
                rows += len(batch); files += 1; batch = []
        if batch:
            pq.write_table(pa.Table.from_pylist(batch),
                           f"{base}/part-{files:05d}.parquet", compression="zstd")
            rows += len(batch); files += 1

    return {"run_id": run_id, "table": table, "rows": rows, "files": files, "path": ba
```

Stream, do not fetchall

`stream_results=True` plus `yield_per` is the difference between a job that handles 50M rows on a 4 GB worker and a job that OOMs. The same applies to `pandas.read_sql` — always pass `chunksize`.

Pattern 2: incremental extract by watermark

Pull only rows changed since the last run, using an `updated_at` column or a monotonically increasing id.

PYTHON

```python
def incremental_extract(table: str, cursor_col: str, state_store) -> dict:
    last = state_store.get(f"{table}.cursor") or "1970-01-01T00:00:00"
    # Overlap window: re-pull a few minutes to tolerate clock skew and long transactio
    # Downstream MERGE makes the duplicates harmless.
    low = (dt.datetime.fromisoformat(last) - dt.timedelta(minutes=15)).isoformat()

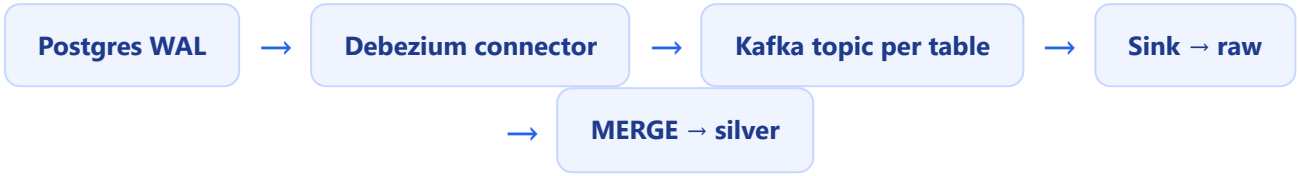
    sql = sa.text(f"""
        select * from {table}
        where {cursor_col} > :low
        order by {cursor_col}
    """)
    rows = list(engine.connect().execution_options(stream_results=True).execute(sql, {
    if rows:
        new_cursor = max(r._mapping[cursor_col] for r in rows).isoformat()
        write_raw(table, rows)
        state_store.set(f"{table}.cursor", new_cursor)   # only AFTER a successful wri
    return {"rows": len(rows), "from": low}
```

Three ways watermarks silently lose data

1. Hard deletes. A deleted row has no `updated_at` change, so it stays in your warehouse forever. **2. Long transactions.** A row written at 10:00 but committed at 10:20 with its original timestamp is skipped if your cursor already passed 10:00 — this is why you need the overlap window, and why reading from a replica makes it worse. **3. Backdated updates.** A correction job sets `updated_at` to the original order date; your cursor never sees it. Mitigate with a weekly full reconciliation that compares row counts and checksums per partition.

Pattern 3: change data capture (CDC)

CDC reads the database's write-ahead log directly, so it sees inserts, updates *and* deletes, in commit order, with near-zero load on the source. It is the correct answer when the source is a relational database and the customer needs freshness under an hour or cares about deletes.



SQL

```sql
-- Postgres prerequisites (the DBA must do this; expect a negotiation)
ALTER SYSTEM SET wal_level = 'logical';      -- requires a restart
-- REPLICA IDENTITY FULL makes UPDATE/DELETE events carry the full "before" image.
-- Without it you get only the primary key, which breaks many downstream merges.
ALTER TABLE public.orders REPLICA IDENTITY FULL;

CREATE PUBLICATION dbz_pub FOR TABLE public.orders, public.customers;
CREATE ROLE debezium WITH REPLICATION LOGIN PASSWORD '...';
GRANT SELECT ON public.orders, public.customers TO debezium;
```

JSON

```
{
  "name": "orders-cdc",
  "config": {
    "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
    "database.hostname": "pg-primary.internal",
    "database.port": "5432",
    "database.user": "debezium",
    "database.password": "${file:/secrets/dbz.properties:password}",
    "database.dbname": "appdb",
    "topic.prefix": "prod",
    "plugin.name": "pgoutput",
    "publication.name": "dbz_pub",
    "slot.name": "dbz_orders",
    "table.include.list": "public.orders,public.customers",
    "snapshot.mode": "initial",
    "heartbeat.interval.ms": "10000",
    "decimal.handling.mode": "string",
    "time.precision.mode": "connect",
    "tombstones.on.delete": "true",
    "key.converter": "org.apache.kafka.connect.json.JsonConverter",
    "value.converter": "io.confluent.connect.avro.AvroConverter",
    "value.converter.schema.registry.url": "http://schema-registry:8081"
  }
}
```

A Debezium event looks like this. The `before` / `after` envelope is what makes deletes and audit trails possible:

JSON

```
{
  "op": "u",
  "ts_ms": 1763462400123,
  "before": {"order_id": 9912, "status": "PAID",    "amount": "199.00"},
  "after":  {"order_id": 9912, "status": "SHIPPED", "amount": "199.00"},
  "source": {"db": "appdb", "schema": "public", "table": "orders",
             "lsn": 26928453, "txId": 884123, "ts_ms": 1763462400001}
}
```

The replication slot will page you at 03:00

If your CDC consumer stops, Postgres **cannot recycle WAL segments** that the slot has not confirmed. Disk fills. The database goes read-only or crashes. This is a production outage caused by your pipeline, and it is the fastest way to destroy trust at a customer. Mitigations: alert on `pg_replication_slots.confirmed_flush_lsn` lag in bytes with a threshold well below free disk; set `max_slot_wal_keep_size` so Postgres sacrifices the slot instead of itself; and have a documented procedure for dropping and re-snapshotting a slot.

SQL

```sql
-- Put this in your monitoring. Alert at a few GB, page at tens of GB.
select slot_name,
       active,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn)) as l
from pg_replication_slots
order by pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn) desc;
```

Pattern 4: API pulls

Every SaaS API is different and all of them will hurt you in one of four ways: pagination, rate limits, auth expiry, and silent schema drift. Write one hardened client and reuse it.

PYTHON

```
import time, logging, random
import httpx
from tenacity import retry, stop_after_attempt, wait_exponential_jitter, retry_if_exce

log = logging.getLogger(__name__)

class RetryableHTTP(Exception): pass

@retry(stop=stop_after_attempt(6),
       wait=wait_exponential_jitter(initial=1, max=60),
       retry=retry_if_exception_type(RetryableHTTP),
       reraise=True)
def _get(client: httpx.Client, url: str, **kw) -> httpx.Response:
    r = client.get(url, **kw)
    if r.status_code == 429:
        # Honour the server. Retry-After may be seconds or an HTTP date.
        wait = float(r.headers.get("Retry-After", 5))
        log.warning("rate limited, sleeping %.1fs", wait)
        time.sleep(wait + random.uniform(0, 1))
        raise RetryableHTTP("429")
    if 500 ≤ r.status_code < 600:
        raise RetryableHTTP(f"{r.status_code}")
    r.raise_for_status()
    return r

def paginate_cursor(base_url: str, token: str, page_size: int = 500):
    """Cursor pagination - the only kind that is safe while data is changing."""
    with httpx.Client(timeout=httpx.Timeout(30.0, connect=10.0),
                      headers={"Authorization": f"Bearer {token}"}) as client:
        cursor, pages = None, 0
        while True:
            params = {"limit": page_size}
            if cursor:
                params["cursor"] = cursor
            data = _get(client, base_url, params=params).json()
            yield from data["results"]
            cursor = data.get("next_cursor")
            pages += 1
            if not cursor:
                break
            if pages > 10_000:
                raise RuntimeError("pagination did not terminate - check next_cursor s
```

⚠ **Offset pagination is a data-loss bug**

`?offset=1000&limit=100` over a table that is being inserted into will skip and duplicate rows as the underlying result set shifts. If the API only offers offset pagination, either freeze the result with a server-side filter (`updated_at < run_start_time`) or accept that you must reconcile. Always prefer cursor/keyset pagination.

Pattern 5: file drops (SFTP, S3, email)

Still extremely common in enterprise, especially with partners, banks and logistics providers. The operational issues are predictable, so build for them once.

Failure	Symptom	Defence
Partial file picked up	Half a CSV ingested; counts wrong	Only process files with a matching `.done` / `.ok` marker, or require atomic rename from a temp name
File re-dropped	Duplicate rows downstream	Record a content hash of every processed file; skip hashes already seen
No file arrives	Silently stale data for days	A freshness sensor that fails loudly at the SLA boundary — absence must be an alert
Header changed	Columns silently shift; garbage values	Assert the exact header list before parsing; fail the batch, do not coerce
Encoding/BOM/delimiter surprise	Mojibake, one giant column	Pin encoding explicitly; sniff and assert the dialect; never rely on defaults
Timezone-naive timestamps	Off-by-5:30 errors in India, off-by-one-day at month end	Demand the timezone in the interface contract; store everything as UTC with the original offset preserved

PYTHON

```python
import hashlib, csv, io, gzip, pathlib

EXPECTED_HEADER = ["order_id", "order_ts", "customer_id", "amount", "currency", "statu

def ingest_drop(path: pathlib.Path, seen_hashes: set[str]) -> dict:
    raw = path.read_bytes()
    digest = hashlib.sha256(raw).hexdigest()
    if digest in seen_hashes:
        return {"status": "skipped_duplicate", "sha256": digest}

    text = gzip.decompress(raw).decode("utf-8-sig") if path.suffix == ".gz" \
           else raw.decode("utf-8-sig")              # utf-8-sig strips the BOM Excel

    reader = csv.reader(io.StringIO(text))
    header = next(reader)
    if header ≠ EXPECTED_HEADER:
        raise ValueError(
            f"header drift in {path.name}\n  expected: {EXPECTED_HEADER}\n  got:
        )

    rows = list(reader)
    write_raw_bytes(f"raw/partner_x/orders/ingest_date={today()}/{digest}.csv.gz", raw
    seen_hashes.add(digest)
    return {"status": "ok", "rows": len(rows), "sha256": digest}
```

Choosing a pattern

Situation	Pattern	Typical freshness
Small dimension table, < 5M rows	Full extract, overwrite	Daily / hourly
Large append-only fact table	Incremental by timestamp or id	Hourly / 15 min
Mutable table where deletes matter	CDC (Debezium / native connector)	Seconds to minutes
SaaS source (Salesforce, HubSpot, Stripe)	Managed connector (Fivetran/Airbyte) unless cost forbids	15 min - daily
Partner / regulator / bank	File drop + marker file + hash dedupe	Daily
Product telemetry	Event stream → Kafka/Kinesis → object storage	Seconds

✦ **Build vs buy, honestly**

A managed connector costs real money per monthly active row but saves weeks per source and removes a maintenance burden your customer must carry after you leave. Custom code is cheaper at the margin and infinitely flexible but is *yours* forever. The defensible split: buy connectors for commodity SaaS sources with stable APIs; build for the customer's proprietary systems, the mainframe, and anything where a vendor's pricing model punishes high-churn tables.

ELT vs ETL and Why ELT Won

For twenty years, transformation happened in a dedicated server between source and warehouse, because warehouse compute was scarce and expensive. Cloud warehouses inverted that: compute is elastic and storage is cheap, so it became better to load raw data first and transform inside the warehouse with SQL. That one inversion is the reason the modern data stack looks the way it does.

ETL (EXTRACT → TRANSFORM → LOAD)

Transform in Informatica/SSIS/Talend/custom Spark *before* loading. The warehouse only ever sees clean, modelled data. Transformation logic lives in a proprietary tool, often in a GUI, often not in version control. Changing a rule means reprocessing from source, which you may no longer have.

ELT (EXTRACT → LOAD → TRANSFORM)

Load raw into the warehouse, then transform with SQL in place. Logic is SQL files in git, reviewed in pull requests, tested in CI. Raw is retained so you can rebuild any downstream model at will. Costs more storage and more warehouse compute — both of which are cheap relative to engineer time.

Dimension	ETL	ELT
Where logic lives	Proprietary tool, often GUI	SQL/Python in git
Code review	Rare, hard	Normal pull-request flow
Testing	Manual, in a dev environment	Automated (`dbt test`) in CI on every PR
Reprocessing history	Requires re-extraction from source	`dbt run --full-refresh` against retained raw
Who can contribute	Specialist ETL developers	Anyone who writes SQL, including analysts
Cost profile	Fixed server licences + infra	Variable warehouse compute; can surprise you
PII handling	Can mask before landing	Raw PII lands in the warehouse — needs policy and masking

⚠ **The two places ETL still wins**

1. Regulated data you must not land. If card numbers or health identifiers may not enter the analytics platform at all, you tokenise or drop them in flight — that is ETL and it is correct. **2. Very expensive source-side filtering.** If a source emits 10 TB/day and you only need 2%, filtering before load is simple economics. Everything else: load first.

ℹ **What you will actually say to a customer with Informatica**

Do not propose a rip-and-replace; those projects run for years and get cancelled. Propose a **strangler** pattern: new pipelines go to ELT, and existing Informatica jobs are migrated only when they need substantive change. Within 18 months the proportion flips naturally, and nobody ever has to approve a scary big-bang migration.

Data Modelling

Modelling is the part customers under-value and later blame you for. Badly modelled data produces two dashboards with different revenue numbers, and that single fact destroys trust in the entire platform. The modelling choices are well understood and a century old in accounting terms; you mostly need to know when to apply which one, and how to handle the one genuinely hard problem — history.

Normalisation (3NF) — why OLTP looks the way it does

Third normal form means every non-key attribute depends on the key, the whole key, and nothing but the key. It minimises redundancy, which minimises update anomalies. That is exactly right for a system taking orders and exactly wrong for a system answering questions, because answering a question requires joining eight tables.

Kimball dimensional modelling — the default for analytics

Split the world into **facts** (things that happened: measurements, events, transactions — long and narrow, append-mostly) and **dimensions** (the context: who, what, where, when — wide, relatively small, slowly changing). Arrange them in a **star schema**: one fact table in the middle, dimension tables around it, each one join away.



One join from fact to any dimension. Predictable performance, predictable SQL, BI tools understand it natively.

Grain — declare it before you write a line of SQL

The grain is the answer to "what does one row of this table mean?" It must be a single, unambiguous sentence. "One row per order line per day" is a grain. "Orders data" is not.

The classic fan-out bug

You join a fact at order-line grain to a shipments table at shipment grain. An order with 3 lines shipped in 2 parcels now produces 6 rows, and `sum(order_amount)` is double the truth. Finance notices in the board meeting. The defence: write the grain as a comment at the top of every model, add a uniqueness test on the grain columns, and never join two facts directly — join both to conformed dimensions, or aggregate one to the other's grain first.

Fact table types

Type	Grain	Example	Notes
Transaction fact	One row per event	`fct_order_lines`	Most common; append-only; fully additive measures
Periodic snapshot	One row per entity per period	`fct_daily_inventory`	Measures are semi-additive — sum across products, never across days
Accumulating snapshot	One row per process instance, updated as it progresses	`fct_order_fulfilment` with `ordered_at`, `picked_at`, `shipped_at`, `delivered_at`	Rows are updated in place; excellent for cycle-time analysis
Factless fact	One row per occurrence, no measures	`fct_student_attendance`	Counting rows is the measure; also used for coverage/"what did not happen"

Slowly changing dimensions

A customer moves from the "SMB" segment to "Enterprise". What should last year's reports show? There is no universally right answer, which is why there are types.

Type	Behaviour	History	Use when
Type 0	Never changes after insert	Original only	Date of birth, original signup source
Type 1	Overwrite in place	None	Correcting typos; attributes where history is meaningless (email formatting)
Type 2	New row with `valid_from` / `valid_to` / `is_current` and a new surrogate key	Full	The default for anything that affects reporting: segment, region, price tier, manager
Type 3	Add a `previous_x` column	One step back	Rare; a single reorganisation where people want "old region" and "new region" side by side
Type 4	Current table + separate history table	Full, split	Very wide or very hot dimensions where current lookups must stay small
Type 6	Type 2 rows plus Type 1 "current value" columns on every row	Both views	When users want both "as it was" and "as it is now" without a second join

```SQL
```

```sql
-- SCD Type 2 merge. This is the pattern you will write dozens of times.
-- Target: dim_customer (customer_sk surrogate, customer_id natural key)
MERGE INTO dim_customer AS t
USING (
    -- Rows that are genuinely new or genuinely changed
    SELECT s.customer_id, s.name, s.segment, s.country, s.updated_at
    FROM   stg_customers s
    LEFT JOIN dim_customer d
           ON d.customer_id = s.customer_id
          AND d.is_current
    WHERE d.customer_sk IS NULL
       OR d.name    IS DISTINCT FROM s.name
       OR d.segment IS DISTINCT FROM s.segment
       OR d.country IS DISTINCT FROM s.country
) AS src
ON t.customer_id = src.customer_id AND t.is_current

-- 1) Close the currently-open row
WHEN MATCHED THEN UPDATE SET
    t.valid_to   = src.updated_at,
    t.is_current = FALSE

-- 2) Insert the brand-new natural keys
WHEN NOT MATCHED THEN INSERT
    (customer_sk, customer_id, name, segment, country, valid_from, valid_to, is_curren
VALUES
    (hash(src.customer_id, src.updated_at), src.customer_id, src.name, src.segment,
     src.country, src.updated_at, TIMESTAMP '9999-12-31 00:00:00', TRUE);

-- 3) A second pass inserts the new versions of the rows closed in step 1.
--    (MERGE cannot both close and open a row for the same key in one statement
--     on most engines - this is why dbt snapshots exist.)
```

Use an end date of 9999-12-31, not NULL

With a sentinel high date, point-in-time joins are a clean `BETWEEN`. With NULL you need `(valid_to IS NULL OR ts < valid_to)` in every query, and someone will forget it. Also keep an explicit `is_current` boolean for the common "current state" query — it is redundant, and it is worth it.

SQL

```sql
-- Point-in-time join: what was the customer's segment WHEN the order was placed?
SELECT f.order_id,
       f.order_ts,
       f.amount,
       d.segment      AS segment_at_order_time,
       c.segment      AS segment_today
FROM   fct_orders f
JOIN   dim_customer d
       ON  d.customer_id = f.customer_id
       AND f.order_ts ≥ d.valid_from
       AND f.order_ts <  d.valid_to          -- half-open interval; no overlap, no gap
JOIN   dim_customer c
       ON  c.customer_id = f.customer_id
       AND c.is_current;
```

One Big Table (OBT) and wide denormalised marts

Modern columnar engines make joins cheaper than they used to be, and BI tools do badly with complex star schemas in the hands of non-technical users. So a common final layer is a single very wide table per subject area with every dimension attribute flattened in.

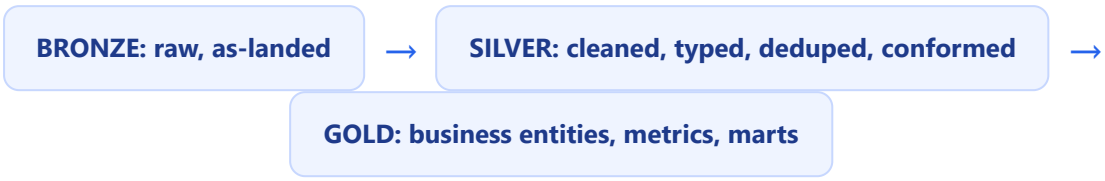
	Star schema	One Big Table
Storage	Minimal duplication	Heavy duplication — but compression absorbs most of it
Query simplicity	Requires correct joins	Trivial: `select … where …`
Self-service safety	Users can fan out facts	Hard to get wrong
Changing a dimension attribute	Update one dimension row	Rebuild the mart
Historical accuracy	Natural via SCD2	Must bake the as-of values in at build time

✓ **What good looks like in practice**

Model the core in a star schema (governed, DRY, correct), then materialise one or two OBTs on top as the consumption layer for BI and for feeding ML/RAG. You keep correctness where it matters and convenience where users live. This is the architecture almost every mature dbt project converges on.

The medallion architecture

Bronze/silver/gold is just a naming convention for layering, but it is a very useful one because it gives you a shared vocabulary with the customer about what is and is not trustworthy.



BRONZE

Exact copy of source, append-only, schema-on-read, PII intact, retained for years. Nobody but pipelines reads it. Columns: the source payload plus `_ingested_at`, `_source_file`, `_batch_id`.

SILVER

Typed, deduplicated on a business key, timezone-normalised to UTC, light conformance (country codes, currency codes), SCD2 applied to dimensions. Tested. This is where data quality lives. Engineers and advanced analysts read it.

GOLD

Business-facing: facts, dimensions, metric tables, OBTs. Named in business language (`net_revenue`, not `amt_net_fin`). Documented, SLA'd, owned. This is the only layer the business should ever point a dashboard at.

⚠ **The layer that gets skipped**

Under schedule pressure, teams build bronze → gold and skip silver. It works for two months. Then every gold model contains its own copy of the deduplication logic, three of them disagree, and fixing a source quirk means editing eleven files. Silver is where you pay once for correctness. Defend it.

SQL for Data Engineers

Application SQL and analytical SQL are different dialects of the same language. Application SQL is about correctness under concurrency; analytical SQL is about expressing a windowed, ranked, period-over-period question in a way that reads clearly and scans as little as possible. The handful of constructs below cover the overwhelming majority of real pipeline code.

CTEs: write queries that someone can review

A nested subquery three levels deep is unreviewable. A chain of named CTEs is a readable program. Modern engines inline CTEs, so the historical performance objection no longer applies on Snowflake, BigQuery, Databricks or recent Postgres (use `MATERIALIZED` / `NOT MATERIALIZED` hints in Postgres if you need to force it).

SQL

```sql
with orders as (
    select * from {{ ref('stg_orders') }}
    where order_status not in ('TEST', 'VOID')
),

refunds as (
    select order_id, sum(refund_amount) as refund_amount
    from {{ ref('stg_refunds') }}
    group by 1
),

joined as (
    select
        o.order_id,
        o.customer_id,
        o.order_ts,
        o.gross_amount,
        coalesce(r.refund_amount, 0)                    as refund_amount,
        o.gross_amount - coalesce(r.refund_amount, 0)   as net_amount
    from orders o
    left join refunds r using (order_id)
)

select * from joined
```

Window functions: the single highest-value skill

Window functions compute across a set of rows related to the current row, without collapsing them. If you are writing a self-join to find "the previous order", you want a window function.

SQL

```sql
select
    order_id,
    customer_id,
    order_ts,
    amount,

    -- Ranking within a partition
    row_number() over (partition by customer_id order by order_ts)           as order
    rank()       over (partition by customer_id order by amount desc)        as amoun
    dense_rank() over (partition by customer_id order by amount desc)        as amoun
    ntile(10)    over (order by amount)                                      as amoun

    -- Looking at neighbours
    lag(order_ts)  over (partition by customer_id order by order_ts)         as prev_
    lead(order_ts) over (partition by customer_id order by order_ts)         as next_
    datediff('day',
        lag(order_ts) over (partition by customer_id order by order_ts),
        order_ts)                                                            as days_

    -- Running aggregates (frame: start of partition to current row)
    sum(amount) over (partition by customer_id order by order_ts
                      rows between unbounded preceding and current row)      as lifet

    -- Moving window (frame: the 6 preceding rows plus this one)
    avg(amount) over (partition by customer_id order by order_ts
                      rows between 6 preceding and current row)              as avg_l

    -- Partition-wide aggregate repeated on every row
    count(*)    over (partition by customer_id)                              as total
    amount / nullif(sum(amount) over (partition by customer_id), 0)          as pct_o

    -- First/last value within the partition
    first_value(amount) over (partition by customer_id order by order_ts
                              rows between unbounded preceding and unbounded following

from fct_orders;
```

The frame clause default will bite you

If you write `order by` inside `over()` without a frame clause, most engines default to `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. `RANGE` treats **ties as a single group**, so two orders with the same timestamp both get the full running total including each other. `ROWS` is positional and almost always what you meant. Be explicit: write the frame clause every time.

Deduplication: the `QUALIFY` pattern

You have a CDC feed with multiple versions of each row and you want the latest. This is the most-written query in data engineering.

SQL

```sql
-- Snowflake, Databricks, BigQuery, DuckDB: QUALIFY filters on window results directly
select *
from raw_orders
qualify row_number() over (
    partition by order_id
    order by updated_at desc, _ingested_at desc, _file_seq desc   -- deterministic tie
) = 1;

-- Portable version for engines without QUALIFY (Postgres, Redshift, MySQL 8)
select * from (
    select *,
           row_number() over (partition by order_id
                              order by updated_at desc, _ingested_at desc) as rn
    from raw_orders
) t
where rn = 1;
```

Always add a deterministic tiebreak

`order by updated_at desc` alone is non-deterministic when two rows share a timestamp — and CDC produces ties constantly. Your pipeline will then produce different results on reruns, which makes every reconciliation argument unwinnable. Add a monotonic secondary key: LSN, Kafka offset, file sequence, or ingest timestamp.

Incremental merge / upsert patterns

Four shapes cover nearly every incremental load. Know which one you are doing and why.

SQL

```sql
-- 1) APPEND ONLY - immutable events. Cheapest; just insert new partitions.
insert into fct_events
select * from stg_events
where event_date = '2025-11-18';

-- 2) INSERT-OVERWRITE BY PARTITION - idempotent reprocessing of a window.
--    Safest batch pattern: rerunning the same day produces the same result.
delete from fct_orders where order_date between '2025-11-15' and '2025-11-18';
insert into fct_orders
select * from stg_orders where order_date between '2025-11-15' and '2025-11-18';
-- Snowflake/Databricks/Spark can do this atomically:
--   insert overwrite table fct_orders partition (order_date) select ...

-- 3) MERGE / UPSERT - mutable rows keyed by a business key.
merge into fct_orders t
using (
    select * from stg_orders
    qualify row_number() over (partition by order_id order by updated_at desc) = 1
) s
on t.order_id = s.order_id
when matched and s.updated_at > t.updated_at then update set
    t.status      = s.status,
    t.amount      = s.amount,
    t.updated_at  = s.updated_at,
    t._merged_at  = current_timestamp()
when not matched then insert
    (order_id, customer_id, order_ts, amount, status, updated_at, _merged_at)
values
    (s.order_id, s.customer_id, s.order_ts, s.amount, s.status, s.updated_at, current_

-- 4) SCD2 APPEND - never update; always add a new version row.
--    Readers filter with is_current or a point-in-time BETWEEN.
```

MERGE requires a deduplicated source

If `stg_orders` contains two rows for `order_id = 99`, Snowflake raises "*Duplicate row detected during DML action*" and Databricks raises a similar error. This failure appears in production the first time a source re-sends a batch. Always dedupe in the `USING` clause, as shown above. The alternative — setting `ERROR_ON_NONDETERMINISTIC_MERGE = FALSE` — silently picks an arbitrary row and is almost always wrong.

Worked example: sessionisation

A classic request: group a user's events into sessions, where a new session starts after 30 minutes of inactivity. This combines `lag`, a boolean flag, and a running sum — a pattern worth memorising because it generalises to any "group consecutive runs" problem.

SQL

```sql
with events as (
    select user_id, event_ts, event_name
    from fct_page_views
    where event_ts ≥ dateadd(day, -7, current_date())
),

flagged as (
    select *,
           lag(event_ts) over (partition by user_id order by event_ts) as prev_ts,
           case
             when lag(event_ts) over (partition by user_id order by event_ts) is null
             when datediff('minute',
                           lag(event_ts) over (partition by user_id order by event_ts)
                           event_ts) > 30 then 1
             else 0
           end as is_session_start
    from events
),

numbered as (
    select *,
           sum(is_session_start) over (partition by user_id order by event_ts
                                       rows between unbounded preceding and current ro
    from flagged
)

select
    user_id,
    user_id || '-' || session_num                as session_id,
    min(event_ts)                                as session_start,
    max(event_ts)                                as session_end,
    datediff('second', min(event_ts), max(event_ts)) as duration_s,
    count(*)                                     as page_views,
    count(distinct event_name)                   as distinct_pages
from numbered
group by user_id, session_num
having count(*) > 1          -- drop single-hit bounces if the business wants that
order by user_id, session_start;
```

Worked example: period-over-period with gap filling

"Show revenue by month with month-over-month growth" is easy until a month has no sales and the growth calculation silently skips it. Always join to a generated date spine.

SQL

```sql
with spine as (
    -- Snowflake; BigQuery: generate_date_array, Postgres: generate_series
    select dateadd(month, seq4(), date_trunc('month', dateadd(year, -2, current_date()
    from table(generator(rowcount => 24))
),

monthly as (
    select date_trunc('month', order_ts) as month_start,
           sum(net_amount)               as revenue,
           count(distinct customer_id)   as customers
    from fct_orders
    group by 1
),

filled as (
    select s.month_start,
           coalesce(m.revenue, 0)   as revenue,
           coalesce(m.customers, 0) as customers
    from spine s
    left join monthly m using (month_start)
)

select
    month_start,
    revenue,
    lag(revenue) over (order by month_start)                                    as pr
    revenue - lag(revenue) over (order by month_start)                          as mo
    round(100.0 * (revenue - lag(revenue) over (order by month_start))
          / nullif(lag(revenue) over (order by month_start), 0), 1)             as mo
    round(avg(revenue) over (order by month_start rows between 2 preceding and current
    sum(revenue) over (partition by year(month_start) order by month_start)     as yt
from filled
order by month_start;
```

SQL habits that separate professionals from amateurs

- [ ] **Never `select *` in a model that writes a table.** Column order changes break downstream inserts; new source columns silently widen your table.

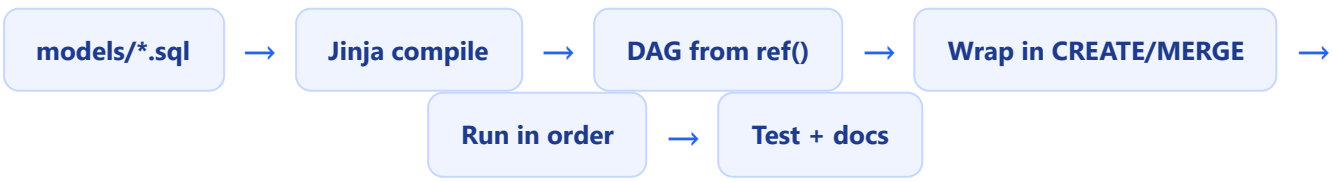
- [ ] **Use `nullif(x, 0)` in every denominator.** Division by zero either errors or produces infinity, and both ruin a dashboard.
- [ ] **Prefer `is distinct from` to `<>` when NULLs are possible.** `NULL <> 'A'` is NULL, not true — this is the single most common change-detection bug.
- [ ] **Cast explicitly.** Implicit casts differ across engines and will change a result when you migrate dialects.
- [ ] **Use `decimal` for money, never `float`.** `0.1 + 0.2 ≠ 0.3`; finance will find it.
- [ ] **Store timestamps in UTC with an explicit type.** Keep the original local time and offset in separate columns if the business reasons in local time.
- [ ] **Alias every table and qualify every column** in any query with more than one table. Unqualified columns become ambiguous the moment a source adds a column.
- [ ] **Put the grain in a comment** at the top of every model file.

dbt: The Transformation Layer

dbt turned SQL transformation into software engineering: version control, code review, automated tests, dependency resolution, documentation and lineage, all generated from SQL files with a templating layer. It is the de-facto standard for the T in ELT, and at a customer site it is frequently the single artefact that most improves how their team works — because it gives analysts the engineering practices they never had.

What dbt actually does

Demystified: dbt reads `.sql` files containing `select` statements, resolves `ref()` calls into a dependency graph, wraps each select in the right DDL for your materialisation, and executes them against your warehouse in topological order. It does not move data. It does not have a compute engine. It is a very good SQL compiler and runner.



Project structure that survives contact with a team

TEXT

```
analytics/
  dbt_project.yml
  packages.yml                  # dbt_utils, dbt_expectations, codegen
  profiles.yml                  # connection (keep OUT of git; use env vars)
  models/
    staging/                    # 1:1 with sources. Rename, cast, light clean. Views.
      _sources.yml              # source definitions + freshness
      _staging__models.yml      # docs + tests for the staging models
      stg_shopify__orders.sql
      stg_shopify__customers.sql
      stg_salesforce__accounts.sql
    intermediate/               # reusable joins/logic. Ephemeral or views. Never expo
      int_orders_with_refunds.sql
      int_customer_first_order.sql
    marts/
      core/
        dim_customers.sql
        dim_products.sql
        fct_orders.sql
      finance/
        fct_revenue_daily.sql
      marketing/
        fct_attribution.sql
  snapshots/
    customers_snapshot.sql      # SCD2 history capture
  tests/
    assert_revenue_reconciles.sql   # singular (bespoke) tests
  macros/
    cents_to_currency.sql
    generate_schema_name.sql
  seeds/
    country_codes.csv           # small static lookups, version controlled
```

Naming conventions that prevent arguments

`stg_<source>__<entity>` (double underscore separates source from entity), `int_<description>`, `dim_<entity>`, `fct_<event>`. One staging model per source table, no exceptions — it gives you exactly one place to fix a source quirk. Decide this in week one and write it in the README; retrofitting naming across 200 models is miserable.

Sources, freshness and staging

YAML

```yaml
# models/staging/_sources.yml
version: 2

sources:
  - name: shopify
    database: raw
    schema: shopify
    loader: fivetran
    meta:
      owner: "data-platform@customer.com"
    freshness:
      warn_after:  {count: 6,  period: hour}
      error_after: {count: 24, period: hour}
    loaded_at_field: _fivetran_synced
    tables:
      - name: orders
        description: "Raw Shopify orders, one row per order. Soft deletes via _fivetra
        columns:
          - name: id
            description: "Shopify order id, primary key."
            tests: [unique, not_null]
          - name: total_price
            description: "Order total in shop currency, as a string."
      - name: customers
        freshness: null          # override: this table legitimately updates rarely
```

SQL

```sql
-- models/staging/stg_shopify__orders.sql
-- Grain: one row per Shopify order.
{{ config(materialized='view') }}

with source as (
    select * from {{ source('shopify', 'orders') }}
    where not coalesce(_fivetran_deleted, false)
),

renamed as (
    select
        id                                          as order_id,
        customer_id,
        cast(created_at as timestamp_ntz)           as ordered_at_utc,
        cast(total_price as decimal(12,2))          as gross_amount,
        upper(trim(currency))                       as currency_code,
        lower(financial_status)                     as payment_status,
        lower(fulfillment_status)                   as fulfilment_status,
        cast(updated_at as timestamp_ntz)           as updated_at_utc,
        _fivetran_synced                            as _loaded_at
    from source
)

select * from renamed
```

Materialisations

Materialisation	What dbt runs	Use when	Cost
`view`	`CREATE OR REPLACE VIEW`	Staging; thin logic; always-fresh needs	Build free, query expensive (re-runs every time)
`table`	`CREATE OR REPLACE TABLE … AS`	Marts under ~100M rows; anything queried often	Build cost each run, query cheap
`incremental`	`MERGE` / `INSERT` of new rows only	Large facts where full rebuild is too slow/expensive	Cheap per run, complex to get right
`ephemeral`	Inlined as a CTE in the consumer	Intermediate logic you never want materialised	Free; but deep chains produce unreadable compiled SQL
`snapshot`	SCD2 merge into a history table	Capturing dimension history from a mutable source	Small; must run on schedule or history is lost
`materialized_view`	Engine-native MV	Warehouse supports it and you want auto-refresh	Continuous background compute

A real incremental model

SQL

```sql
-- models/marts/core/fct_orders.sql
-- Grain: one row per order. Incremental by ordered_at_utc with a 3-day lookback.
{{ config(
    materialized   = 'incremental',
    unique_key     = 'order_id',
    incremental_strategy = 'merge',
    on_schema_change = 'append_new_columns',
    cluster_by     = ['ordered_at_utc'],
    partition_by   = {'field': 'ordered_at_utc', 'data_type': 'timestamp', 'granularit
) }}

with orders as (
    select * from {{ ref('stg_shopify__orders') }}

    {% if is_incremental() %}
      -- Lookback window absorbs late-arriving and backdated rows.
      -- Without it, anything that lands after the watermark is lost forever.
      where ordered_at_utc ≥ (
          select coalesce(dateadd(day, -3, max(ordered_at_utc)), '1900-01-01')
          from {{ this }}
      )
    {% endif %}
),

refunds as (
    select order_id, sum(refund_amount) as refund_amount
    from {{ ref('stg_shopify__refunds') }}
    group by 1
),

customers as (
    select customer_id, customer_sk, segment
    from {{ ref('dim_customers') }}
    where is_current
),

final as (
    select
        o.order_id,
        o.customer_id,
        c.customer_sk,
        c.segment                                       as customer_segment,
        o.ordered_at_utc,
        cast(o.ordered_at_utc as date)                  as order_date,
        o.gross_amount,
        coalesce(r.refund_amount, 0)                    as refund_amount,
        o.gross_amount - coalesce(r.refund_amount, 0)   as net_amount,
        o.currency_code,
```

```
        o.payment_status,
        current_timestamp()                              as _dbt_updated_at
    from orders o
    left join refunds   r on r.order_id    = o.order_id
    left join customers c on c.customer_id = o.customer_id
)

select * from final
```

Three ways incremental models lose data silently

1. No lookback. `where ts > max(ts)` drops every row that arrives late. Always use a lookback window sized to your observed lateness (measure it; do not guess). **2. Joins that change.** If a dimension attribute changes, already-materialised rows keep the old value forever. Either accept that (it is often correct — the value as of order time) or schedule periodic full refreshes. **3.** `on_schema_change` **defaults.** If the default is `ignore`, a new source column silently never appears. Set it explicitly to `append_new_columns` or `sync_all_columns`.

Tests

YAML

```yaml
# models/marts/core/_core__models.yml
version: 2

models:
  - name: fct_orders
    description: |
      One row per order. Net amount is gross less refunds.
      Incremental with a 3-day lookback; full-refresh monthly.
    config:
      contract: {enforced: true}        # dbt enforces column names and types at build
    columns:
      - name: order_id
        data_type: bigint
        description: "Natural key from Shopify."
        tests:
          - unique
          - not_null
      - name: customer_sk
        data_type: varchar
        tests:
          - not_null
          - relationships:
              to: ref('dim_customers')
              field: customer_sk
      - name: net_amount
        data_type: decimal(12,2)
        tests:
          - dbt_utils.accepted_range:
              min_value: -100000
              max_value: 1000000
      - name: payment_status
        data_type: varchar
        tests:
          - accepted_values:
              values: ['paid', 'pending', 'refunded', 'voided']
    tests:
      - dbt_utils.unique_combination_of_columns:
          combination_of_columns: [order_id]
      - dbt_utils.expression_is_true:
          expression: "net_amount ≤ gross_amount"
      - dbt_expectations.expect_table_row_count_to_be_between:
          min_value: 1000
          max_value: 50000000
```

SQL

```sql
-- tests/assert_revenue_reconciles.sql
-- A singular test: passes when it returns ZERO rows.
-- This is how you encode "our mart must agree with the source system".
with mart as (
    select order_date, sum(net_amount) as mart_revenue
    from {{ ref('fct_orders') }}
    where order_date ≥ dateadd(day, -7, current_date())
    group by 1
),
source as (
    select cast(created_at as date) as order_date,
           sum(cast(total_price as decimal(12,2))) - sum(coalesce(refund_total, 0)) as
    from {{ source('shopify', 'orders') }}
    where cast(created_at as date) ≥ dateadd(day, -7, current_date())
    group by 1
)
select m.order_date, m.mart_revenue, s.src_revenue,
       abs(m.mart_revenue - s.src_revenue) as diff
from mart m
join source s using (order_date)
where abs(m.mart_revenue - s.src_revenue) > 0.01
```

Snapshots: SCD2 without writing the merge

SQL

```sql
-- snapshots/customers_snapshot.sql
{% snapshot customers_snapshot %}
{{ config(
    target_schema = 'snapshots',
    unique_key    = 'customer_id',
    strategy      = 'timestamp',
    updated_at    = 'updated_at_utc',
    invalidate_hard_deletes = true
) }}

select customer_id, name, email, segment, country, updated_at_utc
from {{ source('crm', 'customers') }}

{% endsnapshot %}
```

dbt adds `dbt_valid_from`, `dbt_valid_to`, `dbt_scd_id` and manages the merge. Use `strategy='check'` with `check_cols` when the source has no reliable `updated_at`.

Snapshots capture history only while they run

If the snapshot job is paused for a week, the changes in that week are gone — there is no way to recover them from a mutable source. Snapshots must be the *first* thing in your DAG, must run at least as often as the source changes meaningfully, and must be monitored separately from the rest of the pipeline. This is a genuinely unrecoverable failure mode; treat it as such.

Macros

SQL

```sql
-- macros/cents_to_currency.sql
{% macro cents_to_currency(column_name, precision=2) %}
    round(cast({{ column_name }} as decimal(18,4)) / 100, {{ precision }})
{% endmacro %}

-- macros/generate_schema_name.sql
-- Default dbt behaviour concatenates target and custom schema; this overrides
-- it so prod uses clean names and dev is sandboxed per developer.
{% macro generate_schema_name(custom_schema_name, node) -%}
    {%- set default_schema = target.schema -%}
    {%- if target.name == 'prod' and custom_schema_name is not none -%}
        {{ custom_schema_name | trim }}
    {%- elif custom_schema_name is none -%}
        {{ default_schema }}
    {%- else -%}
        {{ default_schema }}_{{ custom_schema_name | trim }}
    {%- endif -%}
{%- endmacro %}
```

Running dbt: the commands that matter

BASH

```
# Install and set up
pip install dbt-core dbt-snowflake      # or dbt-bigquery, dbt-databricks, dbt-postgre
dbt deps                                 # install packages.yml dependencies
dbt debug                                # verify the connection before anything else

# Core workflow
dbt run                                  # build everything
dbt run --select fct_orders              # one model
dbt run --select +fct_orders             # the model and ALL its upstream parents
dbt run --select fct_orders+             # the model and ALL its downstream children
dbt run --select +fct_orders+            # the full lineage both ways
dbt run --select tag:finance             # by tag
dbt run --select state:modified+ --state ./prod_artifacts   # CI: only what changed
dbt run --full-refresh --select fct_orders                  # rebuild an incremental f

dbt test                                 # all tests
dbt build                                # run + test + snapshot + seed, in DAG order
dbt source freshness                     # check source SLAs; fails the job if stale

dbt docs generate && dbt docs serve      # lineage graph + column-level docs
dbt compile --select fct_orders          # see the actual SQL before it runs
cat target/compiled/analytics/models/marts/core/fct_orders.sql
```

✦ **Use `dbt build`, not `dbt run` then `dbt test`**

`build` interleaves tests with model execution in DAG order, so a failing test on `stg_orders` stops the twelve downstream models from being built on bad data. With `run` followed by `test`, the bad data has already propagated to your marts and someone's dashboard before the test fires.

Worked mini-project: a complete dbt deliverable

This is roughly what you should leave behind at a customer after two weeks on a focused scope.

1. **Declare sources with freshness.**
   Every raw table used, with owner metadata and `warn_after` / `error_after` matching the business SLA. This alone creates the first real freshness monitoring they have had.

2. **One staging model per source table.**
   Rename to business language, cast types, trim and upper-case codes, drop soft-deleted rows, convert timestamps to UTC. Views, so they cost nothing to build.

3. **Snapshots for every mutable dimension**

you will need history on — run these first and hourly.

4. **Intermediate models**
for any join or logic used by more than one mart. If logic appears twice, it moves here.

5. **Dimensions and facts**
in `marts/core`, with declared grain, enforced contracts, unique+not_null on keys, and relationship tests on every foreign key.

6. **One wide OBT per consuming team**
in `marts/<team>`, built from core. Teams get a table they cannot misuse.

7. **A reconciliation test**
against the source system for the top one or two metrics. This is the test that ends arguments.

8. **CI on every pull request:**
`dbt build --select state:modified+` against a PR schema, plus SQL linting. No merge without green.

9. **Docs published**
and the lineage graph bookmarked by the analytics team.

10. **A README**
with the naming convention, the layer rules, how to add a source, and how to run CI locally.

Orchestration

An orchestrator answers four questions: what runs, in what order, what happens when something fails, and how do you rerun a window of history. Cron answers the first and none of the others, which is why every customer who started with cron has a folder of shell scripts with `sleep 600` in them to "make sure the previous job finished".

Why cron is not enough

Requirement	Cron	Orchestrator
Dependencies between jobs	`sleep` and hope	Explicit DAG edges
Retry on failure	Write it yourself	Built in, with backoff
Backfill a date range	A bash loop and prayer	First-class, parallel, with concurrency limits
Visibility into what ran	Log files on a box	UI with per-task status, duration, logs, history
Alerting	`|| mail -s`	Callbacks to Slack/PagerDuty with context
Preventing overlapping runs	Lock files, badly	`max_active_runs`, pools, concurrency
Partial rerun from failure	Rerun everything	Clear and rerun only failed tasks and their descendants

Airflow: the concepts that actually matter

DAG AND LOGICAL DATE

A DAG run is identified by its **logical date** (formerly `execution_date`) — the start of the interval the run covers, not when it ran. A daily DAG for 2025-11-18 starts *after* 2025-11-18 ends. Misunderstanding this causes the single most common Airflow bug: off-by-one-day data.

OPERATORS AND SENSORS

An operator is a unit of work (`PythonOperator`, `BashOperator`, `SQLExecuteQueryOperator`). A sensor waits for a condition (a file, a partition, another DAG). Always run sensors in `reschedule` mode or as deferrable operators, or they occupy a worker slot for hours doing nothing.

XCOM AND THE ANTI-PATTERN

XCom passes small values between tasks via the metadata database. It is for identifiers and counts. Pushing a DataFrame through XCom will bloat and eventually break the metadata DB. Pass a path or a key; store the data in object storage.

PYTHON

```python
from __future__ import annotations

import pendulum
from airflow.decorators import dag, task
from airflow.providers.common.sql.operators.sql import SQLExecuteQueryOperator
from airflow.sensors.external_task import ExternalTaskSensor
from airflow.models.baseoperator import chain
from airflow.utils.trigger_rule import TriggerRule

DEFAULT_ARGS = {
    "owner": "data-platform",
    "retries": 3,
    "retry_delay": pendulum.duration(minutes=5),
    "retry_exponential_backoff": True,
    "max_retry_delay": pendulum.duration(minutes=60),
    "email_on_failure": False,          # use on_failure_callback instead; email is no
    "execution_timeout": pendulum.duration(hours=2),
}

@dag(
    dag_id="orders_daily",
    schedule="0 3 * * *",                       # 03:00 UTC
    start_date=pendulum.datetime(2025, 1, 1, tz="UTC"),
    catchup=False,                              # set True only when you WANT historic
    max_active_runs=1,                          # prevents two runs clobbering the sam
    default_args=DEFAULT_ARGS,
    tags=["orders", "tier-1"],
    doc_md=__doc__,
)
def orders_daily():

    wait_for_cdc = ExternalTaskSensor(
        task_id="wait_for_cdc_sync",
        external_dag_id="cdc_sync",
        external_task_id="publish_raw",
        mode="reschedule",                      # frees the worker slot between checks
        poke_interval=300,
        timeout=60 * 60 * 4,
    )

    @task
    def extract(logical_date=None) -> dict:
        """Extract exactly the window [ds, ds+1). Idempotent by construction."""
        ds = logical_date.to_date_string()
        result = run_extract(window_start=ds, window_end=logical_date.add(days=1).to_d
        return {"path": result["path"], "rows": result["rows"], "ds": ds}

    @task
```

```
    def validate(meta: dict) -> dict:
        """Fail loudly and early rather than publishing garbage."""
        if meta["rows"] == 0:
            raise ValueError(f"zero rows extracted for {meta['ds']} - upstream likely
        expected = expected_rows_for(meta["ds"])
        if expected and abs(meta["rows"] - expected) / expected > 0.4:
            raise ValueError(f"row count {meta['rows']} deviates >40% from expected {e
        return meta

    load = SQLExecuteQueryOperator(
        task_id="load_to_silver",
        conn_id="snowflake_default",
        sql="""
            -- Idempotent: delete-then-insert the single partition this run owns.
            delete from silver.orders where order_date = '{{ ds }}';
            insert into silver.orders
            select * from raw.orders_external
            where order_date = '{{ ds }}';
        """,
    )

    @task(trigger_rule=TriggerRule.ALL_SUCCESS)
    def run_dbt(ds=None):
        import subprocess
        subprocess.run(
            ["dbt", "build", "--select", "tag:orders",
             "--vars", f'{{"run_date": "{ds}"}}'],
            check=True, cwd="/opt/analytics",
        )

    @task(trigger_rule=TriggerRule.ONE_FAILED, retries=0)
    def alert_on_failure(**context):
        notify_slack(
            channel="#data-alerts",
            text=f":rotating_light: orders_daily failed for {context['ds']} - "
                 f"{context['dag_run'].get_task_instances()}",
        )

    meta = validate(extract())
    chain(wait_for_cdc, meta, load, run_dbt(), alert_on_failure())

orders_daily()
```

Idempotency: the property everything depends on

A task is idempotent if running it twice with the same parameters produces the same result as running it once. Without this, retries corrupt data and backfills double-count, and you cannot safely operate the

pipeline at all.

Not idempotent	Idempotent
`insert into t select … where date = current_date()`	`delete from t where date = '{{ ds }}'; insert … where date = '{{ ds }}'`
`update counters set n = n + 1`	`merge … when matched then update set n = source.n`
Append to a file	Write to `path/run_id={{ run_id }}/` and publish by swapping a pointer
Read "the last 24 hours" from wall clock	Read the window derived from `{{ data_interval_start }}` and `{{ data_interval_end }}`
Send an email at the end of the task	Send from a separate task guarded by a dedupe key

Never use `current_date()` inside a scheduled task

It makes the task's behaviour depend on when it runs rather than what interval it covers. A retry at 00:05 the next day reads the wrong window; a backfill reads today's data for every historical date. Every date in a task body must derive from the orchestrator's interval parameters. This one rule prevents an entire category of incident.

Backfills

BASH

```bash
# Airflow 2.x
airflow dags backfill orders_daily \
  --start-date 2025-09-01 --end-date 2025-09-30 \
  --reset-dagruns

# Guard rails BEFORE you press enter:
#  - max_active_runs is set (or you will launch 30 concurrent warehouse jobs)
#  - the DAG has a pool with a concurrency limit on the warehouse connection
#  - downstream consumers (reverse-ETL, alerts, emails) are disabled or no-op on backf
#  - you have estimated the warehouse cost: 30 days x per-run credits
#  - you have told the customer, because their dashboards will churn
```

⚠ **The backfill that paged everyone**

A 90-day backfill of a DAG whose last task pushed segments to the marketing platform re-sent 90 days of "abandoned cart" emails to every customer. Always check what is downstream of the DAG you are backfilling, and add a `{{ var('is_backfill', false) }}` guard to any task with a side effect outside the warehouse.

Dagster: assets instead of tasks

Dagster inverts the model. Instead of declaring tasks and edges, you declare **assets** — the tables and files that should exist — and their dependencies. The scheduler works out what to run. This maps much more naturally onto data work, where you care about the state of a table rather than the execution of a script.

PYTHON

```python
import pandas as pd
from dagster import (asset, AssetExecutionContext, Definitions, ScheduleDefinition,
                     define_asset_job, AssetCheckResult, asset_check, MetadataValue,
                     DailyPartitionsDefinition)

daily = DailyPartitionsDefinition(start_date="2025-01-01")

@asset(partitions_def=daily, group_name="orders",
       description="Raw orders extracted from the source Postgres for one day.")
def raw_orders(context: AssetExecutionContext) -> pd.DataFrame:
    day = context.partition_key
    df = extract_orders(day)
    context.add_output_metadata({
        "rows": len(df),
        "day": day,
        "preview": MetadataValue.md(df.head().to_markdown()),
    })
    return df

@asset(partitions_def=daily, group_name="orders")
def clean_orders(raw_orders: pd.DataFrame) -> pd.DataFrame:
    return (raw_orders
            .dropna(subset=["order_id"])
            .drop_duplicates(subset=["order_id"], keep="last")
            .assign(net_amount=lambda d: d.gross_amount - d.refund_amount.fillna(0)))

@asset_check(asset=clean_orders, blocking=True)
def no_negative_amounts(clean_orders: pd.DataFrame) -> AssetCheckResult:
    bad = int((clean_orders.net_amount < 0).sum())
    return AssetCheckResult(passed=bad == 0, metadata={"negative_rows": bad})

orders_job = define_asset_job("orders_job", selection=["raw_orders", "clean_orders"])

defs = Definitions(
    assets=[raw_orders, clean_orders],
    asset_checks=[no_negative_amounts],
    schedules=[ScheduleDefinition(job=orders_job, cron_schedule="0 3 * * *")],
)
```

Choosing an orchestrator

Tool	Model	Strengths	Weaknesses
Airflow	Task DAG	Ubiquitous; enormous provider ecosystem; managed on every cloud (MWAA, Composer, Astronomer); every customer has someone who knows it	Heavy; scheduler/metadata DB to operate; local dev is awkward; logical-date confusion
Dagster	Asset graph	Data-aware lineage, built-in checks, excellent local dev and typing, partitions as a first-class idea	Smaller ecosystem; a conceptual shift for an Airflow team
Prefect	Python flows	Lowest friction — decorate functions; dynamic flows are easy; good hybrid execution model	Weaker lineage story; fewer prebuilt integrations
dbt Cloud / native scheduler	dbt only	Zero extra infrastructure if dbt is all you orchestrate	Cannot orchestrate ingestion or anything non-dbt
Cloud-native (ADF, Step Functions, Workflows)	Managed DAG	No infrastructure; IAM integration; often already paid for	GUI-centric; weaker local testing; vendor lock-in

> **The FDE answer**
> Use what the customer already runs. If they have Airflow, write good Airflow — the marginal value of introducing Dagster rarely exceeds the cost of their team learning it while you are not there. If they have nothing and the team is small and Python-literate, Dagster or Prefect will make them faster. If they have one dbt project and nothing else, dbt Cloud is genuinely sufficient and you should say so rather than building a platform they do not need.

Streaming

Streaming is batch processing where the batch size is one and the job never ends. Everything that is hard about it follows from one fact: in an unbounded stream you can

never be certain that you have seen all the data for a given time window, so every aggregate is a guess that you refine. If a customer asks for streaming, your first job is to find out whether they need seconds or whether "real-time" means "not a day old".

When streaming is actually justified

Use case	Needed latency	Streaming justified?
Fraud blocking at authorisation	< 200 ms	Yes — and it is really an online serving problem, not analytics
Operational dashboard for a logistics control room	Seconds to a minute	Yes
Inventory sync between systems	Minutes	Usually CDC + micro-batch, not true streaming
Personalisation / next-best-action	Seconds	Yes for features; the model itself is often batch-trained
Executive revenue dashboard	Hours to a day	No. Batch. Say so clearly.
Regulatory reporting	Daily / monthly	No, and streaming makes reconciliation harder

⚠ **The cost of "real-time" is mostly operational**
A streaming pipeline runs 24/7, so it has an on-call rotation, state to manage, consumer lag to monitor, schema compatibility to maintain, and no natural "just rerun it tomorrow" recovery. A customer with a three-person data team should not own one unless the business value is unambiguous. Micro-batch every 5-15 minutes gives you 90% of the perceived freshness for 20% of the operational burden, and you can build it with the batch tools they already run.

Kafka in the terms that matter for pipelines

TOPIC AND PARTITION

A topic is an append-only log, split into partitions. **Ordering is guaranteed only within a partition.** If you need per-customer ordering, key by `customer_id` so all of a customer's events land in the same partition. If you key by something random, you have no useful ordering at all.

OFFSET AND CONSUMER GROUP

Each consumer group tracks its own offset per partition. Committing an offset says "I have processed up to here". Commit after writing, not before, or a crash loses data. Partition count caps consumer parallelism — 6 partitions means at most 6 useful consumers in a group.

RETENTION AND COMPACTION

Time/size retention deletes old records (default 7 days). Log compaction instead keeps the latest value per key forever, which turns a topic into a replayable snapshot of current state — ideal for CDC and reference data.

PYTHON

```python
from confluent_kafka import Consumer, KafkaError
import json, signal, time

consumer = Consumer({
    "bootstrap.servers": "kafka-1:9092,kafka-2:9092",
    "group.id": "orders-to-lake",
    "auto.offset.reset": "earliest",
    "enable.auto.commit": False,              # we commit manually, AFTER a durable wr
    "max.poll.interval.ms": 600000,           # must exceed your worst-case batch writ
    "session.timeout.ms": 45000,
    "fetch.min.bytes": 1_000_000,             # batch up for throughput
    "fetch.wait.max.ms": 5000,
})
consumer.subscribe(["prod.public.orders"])

running = True
signal.signal(signal.SIGTERM, lambda *_: globals().__setitem__("running", False))

buffer, last_flush = [], time.time()
BATCH_ROWS, BATCH_SECONDS = 50_000, 60

try:
    while running:
        msgs = consumer.consume(num_messages=5000, timeout=1.0)
        for m in msgs:
            if m.error():
                if m.error().code() == KafkaError._PARTITION_EOF:
                    continue
                raise RuntimeError(m.error())
            buffer.append({
                "payload":   json.loads(m.value()),
                "_partition": m.partition(),
                "_offset":    m.offset(),
                "_kafka_ts":  m.timestamp()[1],
            })

        if buffer and (len(buffer) ≥ BATCH_ROWS or time.time() - last_flush > BATCH_S
            write_parquet_to_s3(buffer)       # must be durable before the next line
            consumer.commit(asynchronous=False)
            buffer, last_flush = [], time.time()
finally:
    if buffer:
        write_parquet_to_s3(buffer)
        consumer.commit(asynchronous=False)
    consumer.close()
```

✦ **Write-then-commit gives you at-least-once**

If the process dies between the write and the commit, you reprocess some messages and write duplicates. That is fine *provided* downstream is idempotent — dedupe on a business key plus Kafka offset, or MERGE on the key. Committing first would give you at-most-once, i.e. silent data loss, which is almost never acceptable. Choose duplicates over loss, then design the dedupe.

Event time vs processing time

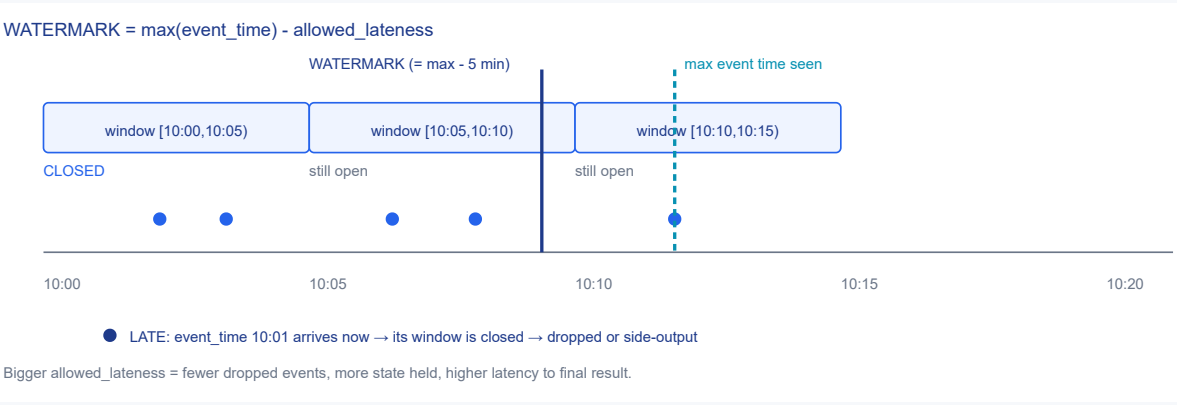
Every streaming bug ultimately comes from confusing these two. **Event time** is when the thing happened (a phone in a tunnel at 10:00). **Processing time** is when your system saw it (10:47, when the phone reconnected). Windowed aggregates must use event time, or your "orders per minute at 10:00" chart is really "orders received per minute", which is a chart of your network.

Windowing

Window	Definition	Example	Use for
Tumbling	Fixed, non-overlapping	Every 5 minutes	Standard periodic aggregates; each event in exactly one window
Sliding / hopping	Fixed size, advancing by a smaller step	10-minute window every 1 minute	Smoothed moving metrics, alerting thresholds
Session	Dynamic; closes after a gap of inactivity	30-minute gap	User sessions, device activity bursts
Global	All time, with explicit triggers	Running totals	Cumulative counters; needs state bounds

Watermarks and late data

A watermark is the engine's assertion: "I believe I have seen all events with event time ≤ T." It is a heuristic, usually "max event time seen minus a slack period". When the watermark passes a window's end, the window is finalised and its state is dropped. Anything that arrives after that is **late**.



The watermark trade-off: completeness versus latency versus state size. There is no setting that gives all three.

PYTHON

```python
from pyspark.sql import functions as F
from pyspark.sql.types import StructType, StructField, StringType, TimestampType, Doub

schema = StructType([
    StructField("order_id",    StringType()),
    StructField("customer_id", StringType()),
    StructField("event_time",  TimestampType()),
    StructField("amount",      DoubleType()),
])

events = (spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "kafka-1:9092")
    .option("subscribe", "orders")
    .option("startingOffsets", "latest")
    .option("maxOffsetsPerTrigger", 500_000)       # cap batch size; prevents a huge f
    .load()
    .select(F.from_json(F.col("value").cast("string"), schema).alias("d"))
    .select("d.*"))

agg = (events
    .withWatermark("event_time", "10 minutes")      # tolerate 10 min of lateness
    .groupBy(
        F.window("event_time", "5 minutes"),
        F.col("customer_id"))
    .agg(
        F.sum("amount").alias("amount_5m"),
        F.count("*").alias("orders_5m"),
        F.approx_count_distinct("order_id").alias("distinct_orders")))

query = (agg.writeStream
    .format("delta")
    .outputMode("append")                           # append is only valid WITH a wate
    .option("checkpointLocation", "s3://lake/_checkpoints/orders_5m")
    .trigger(processingTime="1 minute")             # micro-batch every minute
    .toTable("silver.orders_5m"))

query.awaitTermination()
```

✗ **The checkpoint directory is the job's identity**

Spark Structured Streaming stores offsets and aggregation state there. Delete it and the job restarts from `startingOffsets`, re-reading or skipping data. Point two different queries at the same checkpoint and both corrupt. Change the query's logical plan materially (add a grouping key, change the aggregation) and Spark will refuse to resume — you need a new checkpoint plus a backfill plan. Treat checkpoint paths as versioned, named artefacts, never as scratch.

Delivery semantics

Guarantee	Mechanism	Reality
At most once	Commit offset before processing	Loses data on crash. Acceptable only for metrics you can afford to lose.
At least once	Commit after processing	The practical default. Duplicates on retry; you must dedupe downstream.
Exactly once	Transactional writes + idempotent producers + offset stored atomically with the output	Real within a closed system (Kafka transactions, Flink two-phase commit, Spark + Delta). Across system boundaries it is really "at-least-once plus idempotent sink".

✓ **How to talk about exactly-once with a customer**

Say: "We guarantee at-least-once delivery into the lake, and every write is idempotent on the business key, so the end state is exactly-once. If a message is delivered twice, the MERGE produces the same row." That is true, achievable, and testable — unlike a blanket "exactly-once" claim, which collapses under questioning about external side effects like emails and payments.

Flink vs Spark Structured Streaming

	Flink	Spark Structured Streaming
Model	True record-at-a-time	Micro-batch (continuous mode is limited)
Latency	Tens of milliseconds	Hundreds of ms to seconds
State backend	RocksDB, very large state supported	HDFS/object-store state store; weaker for huge state
Event-time handling	Richer (multiple watermark strategies, side outputs for late data)	Adequate, simpler
Team cost	A specialist skill	Any Spark/PySpark team can run it
Choose when	Sub-second latency, complex event processing, huge keyed state	The customer is already on Spark/Databricks and needs seconds, not milliseconds

Data Quality and Contracts

Every data platform eventually fails the same way: the numbers are wrong and nobody noticed for three weeks. Quality is not a dashboard you build at the end; it is assertions embedded in the pipeline that stop bad data from propagating. The test to apply to any check you write is: would this have caught the last incident, and does it fail loudly enough that someone acts?

The six dimensions, and the check for each

Dimension	Question	Concrete check
Freshness	Is it recent enough?	`max(updated_at) > now() - interval '6 hours'`
Volume	Did roughly the right amount arrive?	Row count within ±40% of the same weekday's 4-week median
Schema	Are the columns and types as expected?	Column set and types asserted; new columns flagged, removed columns fail
Validity	Are values in allowed ranges/sets?	`amount ≥ 0`, `status in (…)`, `email rlike …`
Uniqueness	Is the grain respected?	`count(*) = count(distinct order_id)`
Referential integrity	Do foreign keys resolve?	Every `customer_id` in facts exists in `dim_customers`

✦ **Where to put the check**

Freshness and volume at ingestion, before transformation — they tell you the source is broken. **Schema and validity** at the staging boundary — reject early. **Uniqueness and referential integrity** at the mart boundary — they tell you your modelling is broken. Putting all checks at the end means bad data has already been published.

dbt tests: the 80% solution

If the customer already runs dbt, their quality framework is dbt tests, and adding a second tool is usually a mistake. Generic tests (`unique`, `not_null`, `accepted_values`, `relationships`) plus `dbt_utils` and `dbt_expectations` cover an enormous amount.

YAML

```yaml
models:
  - name: fct_orders
    tests:
      - dbt_utils.recency:
          datepart: hour
          field: ordered_at_utc
          interval: 6
      - dbt_expectations.expect_table_row_count_to_be_between:
          min_value: 5000
          max_value: 500000
      - dbt_expectations.expect_column_values_to_be_within_n_moving_stdevs:
          column_name: daily_row_count
          date_column_name: order_date
          lookback_periods: 1
          trend_periods: 7
          sigma_threshold: 3
    columns:
      - name: order_id
        tests:
          - unique:
              config: {severity: error}
          - not_null
      - name: currency_code
        tests:
          - accepted_values:
              values: ['INR', 'USD', 'EUR', 'GBP', 'AED']
              config: {severity: warn}        # warn: new currencies are expected occa
      - name: customer_id
        tests:
          - relationships:
              to: ref('dim_customers')
              field: customer_id
              config:
                severity: error
                error_if: ">100"              # tolerate a handful of orphans, fail on
                warn_if: ">0"
```

Great Expectations and Soda

When checks must run outside the warehouse — on files at landing, on a Spark DataFrame mid-pipeline, or on a source database before extraction — you need a standalone framework.

PYTHON

```python
import great_expectations as gx

context = gx.get_context()
ds      = context.data_sources.add_pandas("landing")
asset   = ds.add_dataframe_asset("orders_batch")
batch   = asset.add_batch_definition_whole_dataframe("b").get_batch(batch_parameters={

suite = context.suites.add(gx.ExpectationSuite(name="orders_landing"))
suite.add_expectation(gx.expectations.ExpectTableRowCountToBeBetween(min_value=1000, m
suite.add_expectation(gx.expectations.ExpectColumnValuesToNotBeNull(column="order_id")
suite.add_expectation(gx.expectations.ExpectColumnValuesToBeUnique(column="order_id"))
suite.add_expectation(gx.expectations.ExpectColumnValuesToBeBetween(column="amount", m
suite.add_expectation(gx.expectations.ExpectColumnValuesToBeInSet(column="currency", v
suite.add_expectation(gx.expectations.ExpectColumnValuesToMatchRegex(
    column="email", regex=r"^[^@\s]+@[^@\s]+\.[^@\s]+$", mostly=0.99))

result = batch.validate(suite)
if not result.success:
    failed = [r["expectation_config"]["type"] for r in result["results"] if not r["suc
    raise ValueError(f"landing validation failed: {failed}")
```

YAML

```yaml
# Soda: checks as YAML, runs as SQL against the warehouse. Low ceremony, easy handover
checks for fct_orders:
  - row_count between 5000 and 500000
  - missing_count(order_id) = 0
  - duplicate_count(order_id) = 0
  - invalid_percent(currency_code) < 1%:
      valid values: [INR, USD, EUR, GBP]
  - freshness(ordered_at_utc) < 6h
  - avg(net_amount) between 100 and 10000
  - schema:
      fail:
        when required column missing: [order_id, customer_id, net_amount]
        when wrong column type:
          net_amount: decimal
  - anomaly score for row_count < default
```

Schema evolution and registries

Schemas change. The question is whether a change breaks consumers. A schema registry enforces compatibility at publish time, which converts a 3 a.m. production incident into a failed CI build.

Compatibility mode	Allowed change	Meaning
`BACKWARD`	Delete a field; add an **optional** field	New consumers can read old data. The usual default.
`FORWARD`	Add a field; delete an optional field	Old consumers can read new data.
`FULL`	Add or delete **optional** fields only	Both directions. Safest, most restrictive.
`NONE`	Anything	You have no contract. Common and regrettable.

JSON

```
{
  "type": "record",
  "name": "Order",
  "namespace": "com.customer.sales",
  "fields": [
    {"name": "order_id",    "type": "long"},
    {"name": "customer_id", "type": "long"},
    {"name": "amount",      "type": {"type": "bytes", "logicalType": "decimal", "preci
    {"name": "currency",    "type": "string", "default": "INR"},
    {"name": "event_time",  "type": {"type": "long", "logicalType": "timestamp-millis"
    {"name": "channel",     "type": ["null", "string"], "default": null,
     "doc": "Added 2025-11. Nullable with a default, so BACKWARD compatible."}
  ]
}
```

⚠ **The rename trap**

Renaming a field is a delete plus an add. Under BACKWARD compatibility it is allowed, and it silently breaks every consumer that reads the old name — they now get the default. Never rename; add the new field, dual-write both for a deprecation period, migrate consumers, then delete. Use Avro aliases if the registry supports them.

Data contracts

A data contract is an agreement between a producing team and consuming teams: these fields, these types, this grain, this freshness, this is who to call. It converts "the data team's problem" into a shared

obligation, which is the real point — the technology is secondary to the accountability.

YAML

```yaml
# contracts/sales__orders__v1.yaml
apiVersion: v1
kind: DataContract
metadata:
  name: sales.orders
  version: 1.3.0
  owner: sales-platform-team
  slack: "#sales-platform"
  pagerduty: sales-platform-oncall

schema:
  grain: "one row per order"
  primaryKey: [order_id]
  fields:
    - {name: order_id,    type: bigint,        required: true, description: "Immutable
    - {name: customer_id, type: bigint,        required: true, references: "crm.custom
    - {name: ordered_at,  type: timestamp_utc, required: true, description: "UTC. Neve
    - {name: amount,      type: "decimal(12,2)", required: true, constraints: ["≥ 0"]
    - {name: currency,    type: varchar(3),    required: true, enum: [INR, USD, EUR, G
    - {name: status,      type: varchar,       required: true, enum: [new, paid, shipp

serviceLevels:
  freshness:     {threshold: 2h,  measuredBy: "max(ordered_at)"}
  completeness:  {threshold: 99.9, measuredBy: "non-null required fields"}
  availability:  {threshold: 99.5}

changePolicy:
  compatibility: BACKWARD
  noticePeriod: 30d
  breakingChangeProcess: "New major version published in parallel for 90 days"

consumers:
  - {team: finance,   usage: "revenue reporting", criticality: tier-1}
  - {team: marketing, usage: "attribution",       criticality: tier-2}
  - {team: ml,        usage: "churn features",    criticality: tier-2}
```

What makes a contract real

A YAML file nobody enforces is documentation. A contract becomes real when (1) CI fails if the producer's schema diverges from it, (2) the consuming pipeline validates against it at runtime and quarantines violations, and (3) there is a named human and a channel in it. Without all three, you have written a wish.

Quarantine instead of fail-the-whole-batch

PYTHON

```python
def load_with_quarantine(df, contract, run_id: str) -> dict:
    """Split a batch into valid and invalid rather than failing everything.
    One malformed row out of 2 million should not block the pipeline, but it
    must not silently vanish either."""
    valid_mask = (
        df["order_id"].notna()
        & df["amount"].between(0, 1_000_000)
        & df["currency"].isin(contract["currencies"])
        & df["ordered_at"].notna()
    )
    good, bad = df[valid_mask], df[~valid_mask].copy()

    if len(bad):
        bad["_quarantine_reason"] = bad.apply(explain_failure, axis=1)
        bad["_run_id"] = run_id
        write_parquet(bad, f"s3://lake/quarantine/orders/run_id={run_id}/")

    bad_pct = len(bad) / max(len(df), 1)
    if bad_pct > 0.05:                        # a flood means the source changed - stop
        raise ValueError(f"{bad_pct:.1%} of rows quarantined - halting, investigate so

    write_to_silver(good)
    emit_metric("pipeline.quarantined_rows", len(bad), tags={"table": "orders"})
    return {"loaded": len(good), "quarantined": len(bad)}
```

⚠ **Quarantine needs an owner and a review cadence**

A quarantine bucket nobody reads is a slow data-loss mechanism that also makes you feel safe. Put the quarantine count on the main dashboard, alert when it is non-zero for three consecutive days, and name the person who triages it. Review it weekly during the engagement and hand that ritual over explicitly.

Pipeline Reliability

The difference between a pipeline and a production pipeline is entirely in how it behaves when things go wrong — which they will, roughly weekly, because you depend on other

people's systems. Reliability work is unglamorous and it is the reason your platform is still running a year after you leave.

Design for reruns, because you will rerun

- [ ] Every task takes an explicit time window as a parameter; no wall-clock reads.
- [ ] Every write is delete-then-insert by partition, or a MERGE on a business key.
- [ ] Every output path includes a `run_id` until the moment of atomic publication.
- [ ] Every external side effect (email, webhook, reverse-ETL push) is guarded by an idempotency key and skipped during backfills.
- [ ] Every task is safe to kill at any moment — no partial state outside a transaction.

Retries: what to retry and what not to

Error	Retry?	How
Network timeout, connection reset	Yes	Exponential backoff with jitter, 3-6 attempts
HTTP 429 rate limit	Yes	Honour `Retry-After`; add jitter to avoid a thundering herd
HTTP 5xx	Yes	Backoff; cap total time below the task timeout
HTTP 401/403	No	Refresh the token once, then fail loudly — retrying a bad credential can lock the account
HTTP 400, schema violation	No	Fail fast; retrying bad input just wastes time and hides the bug
Out of memory	No	Retrying with the same memory fails the same way. Chunk the work instead.
Warehouse query timeout	Carefully	Retry once on a larger warehouse; more often it needs a query fix
Deadlock / serialisation failure	Yes	Immediate retry with small random delay

PYTHON

```python
import random, time, logging
from functools import wraps

log = logging.getLogger(__name__)

RETRYABLE = (TimeoutError, ConnectionError, TransientBackendError)

def with_retry(max_attempts=5, base=1.0, cap=60.0):
    def deco(fn):
        @wraps(fn)
        def wrapper(*a, **kw):
            for attempt in range(1, max_attempts + 1):
                try:
                    return fn(*a, **kw)
                except RETRYABLE as e:
                    if attempt == max_attempts:
                        log.error("%s failed after %d attempts: %s", fn.__name__, atte
                        raise
                    # Full jitter: prevents synchronised retry storms across many work
                    delay = random.uniform(0, min(cap, base * 2 ** (attempt - 1)))
                    log.warning("%s attempt %d failed (%s); sleeping %.1fs",
                                fn.__name__, attempt, e, delay)
                    time.sleep(delay)
        return wrapper
    return deco
```

Late-arriving data

Measure your lateness distribution before choosing a strategy. Most teams guess and are wrong by an order of magnitude in one direction or the other.

SQL

```sql
-- How late does data actually arrive? Run this before designing your lookback.
select
    datediff('hour', event_ts, _ingested_at)                as lateness_hours,
    count(*)                                                as events,
    round(100.0 * count(*) / sum(count(*)) over (), 3)      as pct,
    round(100.0 * sum(count(*)) over (order by datediff('hour', event_ts, _ingested_at
          / sum(count(*)) over (), 3)                       as cumulative_pct
from bronze.events
where _ingested_at ≥ dateadd(day, -30, current_date())
group by 1
order by 1;
-- Typical result: 99.0% within 1h, 99.8% within 6h, 99.95% within 24h,
-- and a long tail of rows that are 30+ days late from one broken partner integration.
```

LOOKBACK WINDOW

Reprocess the last N days on every run. Simple, idempotent, costs N× the compute of one day. Size N at your p99.9 lateness. The default choice for batch.

LATE SIDE-CHANNEL

Route events past the watermark into a separate table, then merge them during a nightly reconciliation. Keeps the hot path fast and makes lateness visible as a number.

RESTATEMENT

Accept that the last N days are provisional and republish them. Tell consumers explicitly: "figures for the last 3 days may change." This is honest and avoids a doomed quest for immediate perfection.

The monitoring that actually catches incidents

Signal	Alert threshold	Catches
DAG run duration	> 2× the 7-day median	Data volume explosions, warehouse contention, a bad join
Rows written per run	Outside ±40% of same-weekday median	Partial extracts, source outages, duplicate loads
Table freshness	> SLA, checked independently of the DAG	A DAG that is not running at all — which DAG-based alerts cannot catch
Test failures	Any error-severity failure	Grain violations, broken keys, bad values
Quarantine count	> 0 for 3 consecutive days	Slow source drift
Warehouse spend	Daily cost > 1.5× the 30-day mean	Runaway queries, an accidental cross join, a forgotten backfill
CDC replication lag	> a few GB of WAL	The incident that takes the source database down

Alert on absence, not just on failure
The most dangerous failure is a pipeline that stops running. No task fails, so no alert fires, and the dashboard quietly shows last Tuesday's numbers. You need a **heartbeat** check that runs independently of the pipeline — a scheduled query asserting `max(_loaded_at) > now() - SLA` for every tier-1 table, hosted somewhere that does not share fate with the orchestrator.

An incident runbook template worth leaving behind

TEXT

```
PIPELINE: orders_daily          TIER: 1          SLA: data for day D by 07:00 IST on D
OWNER: data-platform            ESCALATION: #data-alerts → on-call → head of data

SYMPTOM: dashboard stale / test failed / DAG failed

1. CHECK SCOPE
   - Airflow UI: which task failed, which logical date?
   - Is it one date or many? One table or all?
   - Run: select max(_loaded_at), count(*) from silver.orders where order_date = '<D>'

2. COMMON CAUSES (in observed frequency order)
   a. Source API 5xx / credential expiry   → check extract task logs for status codes
   b. Schema drift in source               → check the staging test failure message
   c. Warehouse queue / timeout            → check warehouse load; consider a larger
   d. CDC slot lag or disconnection        → check pg_replication_slots
   e. Upstream DAG did not run             → check the ExternalTaskSensor

3. SAFE RECOVERY
   - Clear the failed task and its downstream, then rerun:
       airflow tasks clear orders_daily -t load_to_silver -d -s <D> -e <D>
   - All loads are idempotent (delete+insert by order_date) so reruns are safe.
   - If more than 3 days are affected, use a backfill with max_active_runs=1
     and notify #analytics first (dashboards will churn).

4. IF NOT RESOLVED IN 60 MIN
   - Post in #data-alerts with: affected dates, tables, suspected cause, ETA
   - Escalate to on-call
   - Put a banner on the affected dashboards (do not let people use stale data silentl

5. AFTER
   - Write 5 lines in the incident log: what, why, how long, fix, prevention
   - Add a test that would have caught it earlier
```

Governance, Lineage and Privacy

Governance is where a technically excellent platform gets blocked for six months. The security and privacy teams are not obstacles; they are stakeholders whose approval you need, and they will approve faster if you bring them a design rather than a finished system. At a customer site, the FDE who speaks fluent "data classification and access control" ships; the one who treats it as paperwork does not.

Lineage

Lineage answers two questions that come up constantly: **"where did this number come from?"** (upstream, for debugging and audit) and **"what breaks if I change this?"** (downstream, for impact analysis). Table-level lineage is the minimum; column-level is what actually saves you during a migration.

Source of lineage	Granularity	Effort
dbt `ref()` graph	Table, plus column-level in recent versions	Free if you use dbt
Warehouse query history parsing	Table and often column	Moderate; catches non-dbt SQL too
OpenLineage events from the orchestrator	Job-level, cross-tool	Moderate; the open standard worth betting on
Commercial catalogue (Collibra, Alation, Atlan, Purview, Unity Catalog)	Everything, with ownership and glossary	High cost, high governance value
A hand-drawn diagram	Fiction within six weeks	Low — and it is a trap

✦ **Cheapest real lineage**

If the customer uses dbt, `dbt docs generate` plus the `manifest.json` gives you a complete, always-current dependency graph for free, and you can query the manifest programmatically for impact analysis. Before proposing a $200k catalogue, show them what they already have.

Classifying PII

Class	Examples	Handling
Direct identifiers	Name, email, phone, Aadhaar, PAN, passport, account number	Masked by default; access by explicit grant; tokenise where possible
Quasi-identifiers	Postcode + DOB + gender, device id, precise geolocation	Dangerous in combination — three of these re-identify most people. Generalise or aggregate.
Sensitive categories	Health, biometrics, religion, caste, sexual orientation, political views	Usually needs explicit consent and a separate legal basis; often must not leave the source system
Financial	Card PAN, CVV, bank details	PCI DSS scope — the correct answer is almost always "do not land it at all"
Derived-sensitive	"Pregnancy likelihood score", "financial distress flag"	Treated as sensitive even though no raw PII is present. Teams forget this constantly.

SQL

```sql
-- Snowflake: dynamic data masking. One policy, applied to many columns.
create or replace masking policy mask_email as (val string) returns string ->
  case
    when current_role() in ('ANALYTICS_PII', 'DATA_PLATFORM_ADMIN') then val
    when current_role() in ('ANALYST')
      then regexp_replace(val, '^[^@]+', '***')          -- ***@example.com
    else '**REDACTED**'
  end;

alter table dim_customers modify column email set masking policy mask_email;

-- Row access policy: a sales rep sees only their own region
create or replace row access policy region_rap as (region string) returns boolean ->
  current_role() = 'DATA_PLATFORM_ADMIN'
  or exists (select 1 from governance.user_region_map m
             where m.role_name = current_role() and m.region = region);

alter table fct_orders add row access policy region_rap on (region);

-- Tag-based governance: classify once, enforce everywhere
create tag pii_level allowed_values 'none', 'low', 'high';
alter table dim_customers modify column email  set tag pii_level = 'high';
alter table dim_customers modify column pan    set tag pii_level = 'high';
alter table dim_customers modify column city   set tag pii_level = 'low';

-- Then apply a masking policy to the TAG, so every tagged column is covered,
-- including ones added next year by someone who never read your document.
alter tag pii_level set masking policy mask_high_pii;
```

✓ **Tag-based policy is the one to push for**

Column-by-column policies rot: someone adds `customer_email_2` and it is unprotected. Tagging the column and attaching policy to the tag means governance scales with the platform rather than with your vigilance. Databricks Unity Catalog, BigQuery policy tags and Snowflake tags all support this.

Pseudonymisation vs anonymisation

Technique	Reversible?	Still personal data?	Use for
Hashing (SHA-256 of an email)	Effectively yes — the space is small enough to brute force	**Yes**	Joining across systems; not a privacy control on its own
Salted hashing / HMAC with a secret key	Only with the key	Yes (pseudonymous)	Stable identifiers without exposing the raw value
Tokenisation via a vault	Yes, via the vault	Yes	Payment data, regulated identifiers
Generalisation (DOB → birth year, postcode → district)	No	Reduced risk	Analytics that do not need precision
Aggregation with a minimum cell size	No	Usually not	Published statistics
Differential privacy (calibrated noise)	No	No, with a formal guarantee	External data sharing, research datasets

✕ **Hashing an email is not anonymisation**

There are only so many plausible email addresses, and an attacker with a list can hash all of them and match. Regulators treat a plain hash as pseudonymous personal data, not anonymous. If you need a non-reversible identifier, use HMAC with a key held in a KMS the analytics platform cannot read, and rotate it.

GDPR, India's DPDP Act, and what they demand of a pipeline

Obligation	What it means for your pipeline
Lawful basis / consent	You must be able to say why each dataset exists. Record purpose in the catalogue, per dataset.
Purpose limitation	Data collected for billing may not silently become ML training data. This is a real architectural constraint, not a formality.
Data minimisation	Do not land columns you do not need. "We might want it later" is not a basis.
Right to erasure	You need to find every copy of a person across bronze, silver, gold, exports, backups and model training sets — and delete it, including after table-format vacuum. Design for this on day one; retrofitting is brutal.
Right of access / portability	Produce everything you hold about one person, in a readable format, within a statutory window.
Storage limitation	Retention policies that actually execute. An unenforced policy is worse than none because it is evidence you knew.
Cross-border transfer	DPDP allows transfers except to restricted countries; GDPR needs SCCs or adequacy. Determines which cloud region — and which LLM API — you may use.
Breach notification	Tight windows (72h under GDPR; prescribed timelines under DPDP). You need to know what was in a dataset to report it.

SQL

```sql
-- A practical erasure implementation. The key design choice is a crosswalk
-- table so you can find a subject across every layer without scanning everything.
create table governance.erasure_requests (
    request_id      varchar primary key,
    subject_key     varchar not null,           -- the HMAC pseudonym, not the raw ema
    requested_at    timestamp_ntz not null,
    due_by          timestamp_ntz not null,
    status          varchar not null,           -- received|in_progress|complete|faile
    completed_at    timestamp_ntz,
    tables_affected array
);

-- Execution, per table, driven by a registry of PII-bearing tables
delete from silver.customers      where subject_key = :subject_key;
delete from silver.orders         where subject_key = :subject_key;
update  gold.fct_orders           set customer_sk = '__ERASED__', customer_email = nul
                                  where subject_key = :subject_key;

-- Then, crucially, expire the history that still contains them
vacuum delta.`s3://lake/silver/customers` retain 168 hours;

-- And prove it
select 'silver.customers' as t, count(*) from silver.customers where subject_key = :su
union all
select 'silver.orders',          count(*) from silver.orders    where subject_key = :s
-- Must return zeros. Store this output as evidence against the request_id.
```

⚠ **The questions that stall projects, asked early**

Ask in week one: "Which datasets contain personal data? Where may that data be processed geographically? Who approves a new consumer of a dataset? What is our retention policy and is it enforced anywhere? Have we ever executed an erasure request end to end?" The last one is the most revealing — most organisations have a policy and have never tested it.

Access control that survives your departure

- ☐ Grant to **roles**, never to individuals. People leave; roles persist.
- ☐ Use functional roles (`ANALYST_FINANCE`, `ENGINEER_PLATFORM`) mapped from the customer's identity provider groups, so joiners/leavers are handled by HR processes, not by you.
- ☐ Default deny on raw and bronze. Almost nobody needs raw access; those who think they do usually need a documented silver model.

- [ ] Separate read from write. Analysts read gold; only service principals write.
- [ ] One service principal per pipeline, with the minimum grants that pipeline needs — not one superuser shared by everything.
- [ ] Time-bounded elevated access for debugging, with an audit trail, not a permanent admin grant "just in case".
- [ ] Write the role matrix into a document and hand it over. The matrix, not the grants, is the deliverable.

Performance and Cost

Warehouse bills grow without anyone deciding to grow them. Six months after launch a customer is spending four times the plan and nobody can explain it, because cost is the sum of thousands of small decisions nobody reviewed. Cost control is an engineering discipline, and being the person who cuts a bill by 60% without degrading anything is one of the most reliable ways to earn a renewal.

Where the money actually goes

#1 IDLE / OVERSIZED COMPUTE

#2 FULL SCANS THAT SHOULD BE PRUNED

#3 REBUILDING WHAT COULD BE INCREMENTAL

#4 DASHBOARDS REFRESHING ON A 5-MIN TIMER

Partition pruning: the biggest single lever

Pruning means the engine skips data without reading it. Defeating pruning is the most common performance mistake in customer code, and it is almost always a function wrapped around the partition column.

SQL

```sql
-- DEFEATS PRUNING: the function hides the column from the optimiser.
-- Engine must evaluate date_trunc on every row of every partition.
select count(*) from fct_orders
where date_trunc('month', order_date) = '2025-11-01';

-- PRUNES: a bare range predicate on the partition column.
select count(*) from fct_orders
where order_date ≥ '2025-11-01' and order_date < '2025-12-01';

-- DEFEATS PRUNING: implicit cast on the column side.
select * from fct_orders where cast(order_date as string) like '2025-11%';

-- DEFEATS PRUNING: a non-deterministic or correlated subquery in the predicate
-- can prevent static pruning on some engines; materialise the bound first.
select * from fct_orders
where order_date = (select max(order_date) from fct_orders);   -- check the plan!

-- PRUNES: compute the bound, then pass it as a literal/parameter.
set max_dt = (select max(order_date) from fct_orders);
select * from fct_orders where order_date = $max_dt;
```

✦ **Read the query plan, always**

`EXPLAIN` in Snowflake/Databricks, the execution details pane, or BigQuery's "bytes processed" estimate before you run. The number you want is **partitions scanned / partitions total**. If a query on a 1,000-partition table scans 1,000 partitions to answer a one-day question, you have found a ten-minute fix worth thousands of dollars a year.

Clustering and sort keys

Partitioning handles one dimension (usually date). Clustering handles the second and third. In Snowflake, micro-partitions are automatically created but their usefulness depends on how well-ordered your data is.

SQL

```sql
-- Snowflake: check clustering health before changing anything
select system$clustering_information('fct_orders', '(order_date, customer_id)');
-- Look at average_overlaps and average_depth. Depth near 1 is excellent;
-- depth in the hundreds means the clustering key is doing nothing for you.

alter table fct_orders cluster by (order_date, customer_id);
-- Automatic clustering is a BACKGROUND COST. Enable it only on large tables
-- that are frequently filtered on the key, and watch the credits:
select table_name, sum(credits_used) as credits
from snowflake.account_usage.automatic_clustering_history
where start_time ≥ dateadd(day, -30, current_timestamp())
group by 1 order by credits desc;

-- Databricks: liquid clustering (no partition columns needed)
ALTER TABLE fct_orders CLUSTER BY (order_date, customer_id);
OPTIMIZE fct_orders;

-- BigQuery: partition + cluster declared at creation
CREATE TABLE ds.fct_orders
PARTITION BY DATE(order_ts)
CLUSTER BY customer_id, region
AS SELECT * FROM ds.staging_orders;
```

Warehouse sizing

Snowflake warehouse sizes double in both capacity and cost at each step. The rule people get wrong: a query that takes 8 minutes on Small and 4 minutes on Medium costs **exactly the same**, because you bought twice the compute for half the time. Sizing up is free for perfectly parallel work — and strictly worse for work that does not parallelise.

Size	Credits/hour	Use for
X-Small	1	dbt on small models, BI with caching, development
Small	2	Typical ELT batch
Medium	4	Large joins, heavy transformations
Large	8	Full-refresh rebuilds, large backfills
X-Large+	16+	One-off migrations; rarely justified on a schedule

SQL

```sql
-- Warehouse hygiene that pays for itself in a week
alter warehouse etl_wh set
    auto_suspend = 60              -- seconds. The single highest-ROI setting.
    auto_resume  = true
    min_cluster_count = 1
    max_cluster_count = 3          -- multi-cluster handles CONCURRENCY, not query siz
    scaling_policy = 'ECONOMY'     -- STANDARD spins up faster but costs more
    statement_timeout_in_seconds = 3600
    statement_queued_timeout_in_seconds = 600;

-- Separate warehouses per workload so a runaway analyst query cannot
-- starve the nightly load, and so you can attribute cost.
create warehouse bi_wh   warehouse_size = 'SMALL'  auto_suspend = 60;
create warehouse etl_wh  warehouse_size = 'MEDIUM' auto_suspend = 60;
create warehouse ds_wh   warehouse_size = 'LARGE'  auto_suspend = 300;

-- Hard spend guardrail
create resource monitor monthly_cap with
    credit_quota = 5000
    frequency = MONTHLY
    start_timestamp = immediately
    triggers
        on 75  percent do notify
        on 90  percent do notify
        on 100 percent do suspend
        on 110 percent do suspend_immediate;
alter warehouse ds_wh set resource_monitor = monthly_cap;
```

The four classic money-wasters

1. A warehouse with `auto_suspend` unset or set to hours — it bills while idle, 24/7. **2. A BI tool configured to refresh every 5 minutes** against a source that updates daily — this keeps a warehouse permanently warm. **3. A full-refresh model that should be incremental**, rebuilding 2 TB every night to add 4 GB. **4. A forgotten backfill or a `select *` in a dashboard filter** scanning a fact table on every page load. Check all four in your first week; one of them is almost always present.

A cost audit you can run on day three

SQL

```sql
-- 1. Most expensive queries, with the shape that caused it
select
    query_id,
    left(query_text, 120)                                          as sql_preview,
    user_name,
    warehouse_name,
    round(total_elapsed_time / 1000.0, 1)                          as seconds,
    round(bytes_scanned / power(1024,3), 2)                        as gb_scanned,
    round(partitions_scanned * 1.0 / nullif(partitions_total,0) * 100, 1) as pct_parti
    round(bytes_spilled_to_local_storage / power(1024,3), 2)       as gb_spilled_local,
    round(bytes_spilled_to_remote_storage / power(1024,3), 2)      as gb_spilled_remote
from snowflake.account_usage.query_history
where start_time ≥ dateadd(day, -7, current_timestamp())
  and execution_status = 'SUCCESS'
order by total_elapsed_time desc
limit 50;

-- 2. Queries that scan everything (pruning failures = the fix list)
select left(query_text, 150) as sql_preview, count(*) as runs,
       avg(partitions_scanned * 1.0 / nullif(partitions_total,0)) as avg_scan_ratio,
       sum(bytes_scanned) / power(1024,4) as tb_scanned
from snowflake.account_usage.query_history
where start_time ≥ dateadd(day, -7, current_timestamp())
  and partitions_total > 1000
  and partitions_scanned * 1.0 / nullif(partitions_total,0) > 0.9
group by 1 order by tb_scanned desc limit 25;

-- 3. Idle warehouse burn
with runtime as (
    select warehouse_name,
           sum(credits_used) as credits,
           sum(datediff('second', start_time, end_time)) / 3600.0 as hours_billed
    from snowflake.account_usage.warehouse_metering_history
    where start_time ≥ dateadd(day, -30, current_timestamp())
    group by 1
),
work as (
    select warehouse_name,
           sum(total_elapsed_time) / 1000.0 / 3600.0 as hours_executing
    from snowflake.account_usage.query_history
    where start_time ≥ dateadd(day, -30, current_timestamp())
    group by 1
)
select r.warehouse_name, r.credits, round(r.hours_billed,1) as billed_h,
       round(w.hours_executing,1) as busy_h,
       round(100.0 * w.hours_executing / nullif(r.hours_billed,0), 1) as utilisation_p
from runtime r left join work w using (warehouse_name)
```

```
order by r.credits desc;
-- Utilisation below ~20% means auto_suspend is too long or the warehouse is misused.
```

Spilling: the silent performance killer

When a join or sort does not fit in memory, the engine spills to local disk, and then to remote storage. Remote spill is catastrophic — often a 10-50x slowdown. If you see `bytes_spilled_to_remote_storage > 0`, fix it before anything else.

- [ ] Filter before joining, not after — push predicates into CTEs that feed the join.
- [ ] Select only needed columns; a 40-column join spills where a 6-column join does not.
- [ ] Aggregate before joining when the join is many-to-one and you only need the aggregate.
- [ ] Check for an accidental cross join — a missing join condition shows as an enormous intermediate row count.
- [ ] Only after the above: size the warehouse up, which doubles memory.

The cost conversation with a customer

✓ **Frame it as unit economics, not as a bill**

"We spend $42k/month" invites a panic cut. "We spend $0.0021 per dashboard refresh and $0.14 per daily pipeline run, and 38% of spend is one warehouse at 11% utilisation" invites a decision. Build a chargeback view that attributes credits to warehouse, role and dbt model, publish it monthly, and the conversation stops being adversarial.

Feeding AI and ML

Almost every AI project you are brought in for is, under the hood, a data engineering project with a model at the end. The model is a commodity; the pipeline that keeps its inputs correct, fresh and consistent between training and serving is where the engineering is. If you can make a customer's data reliable, the AI part usually becomes straightforward.

Training/serving skew — the defect that destroys models in production

A model scores 0.91 AUC offline and performs like a coin flip in production. Nine times out of ten the cause is that a feature was computed one way in the training pipeline and a different way at inference time.

Skew source	Example	Fix
Different code paths	Training in SQL, serving in Python with a different null handling	One definition, used by both — a feature store or a shared library
Time leakage	Training uses `customer.lifetime_value` as it is *today* for a prediction made last March	Point-in-time correct joins against SCD2 dimensions
Different freshness	Training on a 30-day-old aggregate, serving on a 1-hour-old one	Match the serving latency in training, or document the gap explicitly
Missing-value handling	Training fills nulls with the column mean; serving fills with 0	Serialise the imputation parameters with the model artefact
Category drift	A new product category appears; the encoder has never seen it	Explicit unknown-category bucket plus a monitoring alert on unseen values

SQL

```sql
-- Point-in-time correct feature extraction. This is THE pattern for training sets.
-- For each label event, compute features using ONLY data available before that moment
with labels as (
    select customer_id, churn_date as as_of_ts, 1 as label
    from churn_events
    union all
    select customer_id, sample_date as as_of_ts, 0 as label
    from negative_samples
),

order_features as (
    select
        l.customer_id,
        l.as_of_ts,
        l.label,
        count(o.order_id)                                          as orders_lifetime,
        count(case when o.order_ts ≥ dateadd(day, -30, l.as_of_ts)
                   then o.order_id end)                            as orders_30d,
        coalesce(sum(case when o.order_ts ≥ dateadd(day, -90, l.as_of_ts)
                          then o.net_amount end), 0)               as spend_90d,
        datediff('day', max(o.order_ts), l.as_of_ts)               as days_since_last_or
    from labels l
    left join fct_orders o
           on o.customer_id = l.customer_id
          and o.order_ts < l.as_of_ts            -- <<< the entire point: strictly bef
    group by 1, 2, 3
),

dim_features as (
    select l.customer_id, l.as_of_ts,
           d.segment, d.country, d.plan_tier
    from labels l
    join dim_customers d
         on  d.customer_id = l.customer_id
         and l.as_of_ts ≥ d.valid_from
         and l.as_of_ts <  d.valid_to          -- attribute AS IT WAS at as_of_ts
)

select f.*, d.segment, d.country, d.plan_tier
from order_features f
join dim_features d using (customer_id, as_of_ts);
```

The leak that makes a model look brilliant

A churn model with 0.99 AUC almost certainly has a leaked feature — something that only exists *because* the customer churned, like `cancellation_reason_filled` or an account status that was updated at churn time. Before celebrating any unusually good result, list every feature and ask "could this value have been different before the event?" The `o.order_ts < l.as_of_ts` predicate above is the mechanical defence, but it does not protect against a column that is itself updated in place.

Feature pipelines and feature stores

OFFLINE STORE

Historical feature values, point-in-time correct, used to build training sets. Usually just tables in the warehouse. The important property is that you can reconstruct any feature as of any timestamp.

ONLINE STORE

Low-latency key-value lookup of the *current* feature vector for one entity. Redis, DynamoDB, Bigtable. Written by the same pipeline that writes offline, which is what prevents skew.

REGISTRY

The definition of each feature — name, owner, source, transformation, freshness. Feast, Tecton, Databricks Feature Store, or a dbt project with conventions. Start with the last one; most customers do not need a feature store.

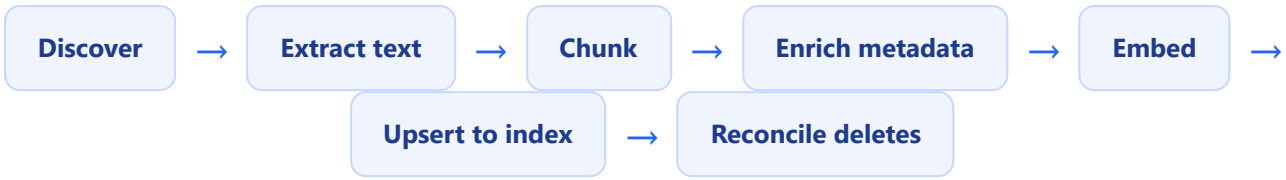
Do they need a feature store?

Almost certainly not in the first engagement. A feature store earns its keep when you have many models sharing many features and genuinely need sub-50 ms online lookups. One model with daily batch scoring needs a well-tested dbt model and a table. Proposing Tecton to a team with two models is how you spend a quarter on infrastructure instead of value.

Document ingestion for RAG at scale

Prototypes load twenty PDFs in a notebook. Production handles a hundred thousand documents that change, with permissions, and must be re-indexed incrementally. The difference is entirely data

engineering.



PYTHON

```python
import hashlib, json
from dataclasses import dataclass, asdict

@dataclass
class Chunk:
    chunk_id: str          # deterministic: hash(doc_id, chunk_index, text)
    doc_id: str
    doc_version: str       # source etag / modified time / content hash
    chunk_index: int
    text: str
    token_count: int
    # Metadata is what makes retrieval filterable - this is where most
    # production RAG quality comes from, far more than the embedding model.
    source_path: str
    title: str
    section: str
    acl_groups: list[str]  # MUST be carried through; filter at query time
    effective_from: str
    effective_to: str | None
    language: str

def chunk_id_for(doc_id: str, idx: int, text: str) -> str:
    h = hashlib.sha256(f"{doc_id}|{idx}|{text}".encode()).hexdigest()[:32]
    return f"{doc_id}:{idx}:{h}"

def incremental_index(docs, vector_store, embedder, state) -> dict:
    """Only embed what changed. Embedding is the expensive step; re-embedding
    100k unchanged documents every night is a pure waste of money and time."""
    seen_doc_ids, embedded, skipped = set(), 0, 0

    for doc in docs:
        seen_doc_ids.add(doc.doc_id)
        prev_version = state.get_version(doc.doc_id)
        if prev_version == doc.version:
            skipped += 1
            continue                                  # unchanged, nothing to do

        chunks = chunk_document(doc)                  # your chunking strategy
        vectors = embedder.embed_batch([c.text for c in chunks], batch_size=256)

        vector_store.delete(filter={"doc_id": doc.doc_id})    # remove old chunks firs
        vector_store.upsert([
            {"id": c.chunk_id, "values": v, "metadata": asdict(c)}
            for c, v in zip(chunks, vectors)
        ])
        state.set_version(doc.doc_id, doc.version)
        embedded += len(chunks)
```

```
    # Deletion reconciliation: documents that disappeared from the source
    # must disappear from the index, or the assistant will cite dead policy.
    for stale_id in state.all_doc_ids() - seen_doc_ids:
        vector_store.delete(filter={"doc_id": stale_id})
        state.delete(stale_id)

    return {"embedded_chunks": embedded, "skipped_docs": skipped}
```

✗ **The two RAG pipeline failures that reach the customer**

1. Permissions are not carried into the index. Every chunk must store the ACL of its source document, and every query must filter on the caller's groups. Otherwise your assistant will cheerfully quote the redundancy plan to the person named in it. **2. Deletions are not reconciled.** A policy document is withdrawn, but its chunks remain in the vector store, so the assistant keeps citing a policy that no longer exists. Both are data-engineering problems, and both are found in demos, not in design reviews.

Embedding pipeline economics

Decision	Impact
Chunk size (256 vs 512 vs 1024 tokens)	Smaller = more chunks = more vectors, more cost, finer retrieval but less context per hit. 400-800 tokens with 10-15% overlap is a sane starting point.
Re-embed on every run vs incremental	100k docs × 5 chunks × re-embedding daily is 150M embeddings/month versus a few thousand. Usually a 100x cost difference.
Batch size	Embedding APIs are far more efficient batched. 128-512 texts per call typically; respect the provider's token limit per request.
Model dimension (384 vs 1536 vs 3072)	Storage and index memory scale linearly. 1M chunks at 1536 float32 dims is ~6 GB before index overhead. Consider dimension reduction or quantisation.
Where embedding runs	A hosted API sends document text to a third party — this is a data-residency and confidentiality question, not a technical one. Check before you build.

Monitoring an ML/AI data pipeline

- ☐ **Feature freshness** per feature, not per table — a stale feature silently degrades predictions.

- [ ] **Feature distribution drift**: PSI or KL divergence against the training distribution, per feature, weekly.
- [ ] **Null rate per feature** — a sudden jump usually means an upstream schema change.
- [ ] **Unseen-category rate** for encoded columns.
- [ ] **Training/serving consistency check**: replay a sample of production inference inputs through the training feature code and assert equality. Run this nightly; it is the only test that actually catches skew.
- [ ] **Index coverage** for RAG: documents in source vs documents indexed vs chunks in the vector store, reconciled daily.

Working With a Customer's Data Team

The technical work is maybe sixty percent of the job. The other forty is getting access, building trust with people who have seen consultants come and go, and leaving behind something their team can actually run. Projects fail far more often on these than on SQL.

Access: start on day zero, expect it to take weeks

Send a single written request with everything you will need, with a justification per item, before your first day. Access requests serialise through security reviews, and each round trip costs days.

TEXT

```
ACCESS REQUEST – [Project] – [Your name] – [Date]

ENVIRONMENT ACCESS
 1. Warehouse READ on raw.* and analytics.*        – to profile sources and validate m
 2. Warehouse WRITE on analytics_dev_<name>        – personal sandbox schema for devel
 3. Git repository (read/write, feature branches)  – to deliver code via pull request
 4. Orchestrator UI (read) + dev project (write)   – to inspect and build DAGs
 5. Object storage read on s3://lake/raw/*         – to inspect landed files
 6. Secrets manager: read on the dev path only     – to run pipelines locally against
 7. Ticketing + Slack/Teams channels               – to work in the open

EXPLICITLY NOT REQUESTED
 – Production write access
 – Access to source transactional systems
 – Any PII in unmasked form (masked views are sufficient for our scope)

DATA
 8. A representative sample (100k rows) of orders, customers, products
    – synthetic or masked is fine; we need realistic distributions, not real people
 9. Current data dictionary / ERD, however out of date
10. Last 3 months of pipeline failure tickets      – to understand real failure modes

ADMINISTRATIVE
11. NDA / DPA status and any data handling constraints we must observe
12. Named approver for each of the above
13. Confirmation of whether data may leave the customer's cloud tenancy

Blocking items: 1, 2, 3. Without these no technical work can start.
```

Never ask for production write access

Ask for read on production and write on a sandbox. It is faster to approve, it removes you as a risk in any incident that happens while you are there, and it signals that you understand their operational reality. When something eventually does need to run in production, it goes through their deployment process — which is also the thing you want them using after you leave.

Sandboxes and sample data

ZERO-COPY CLONE (SNOWFLAKE, BIGQUERY SNAPSHOTS, DELTA SHALLOW CLONE)

Clone production into a dev schema instantly, with no storage cost until you modify it. This is the single best thing to ask for: full-fidelity data, zero risk to production, no copying. `create schema dev_pranit clone analytics;`

MASKED OR SYNTHETIC EXTRACTS

When data may not leave the tenancy or PII cannot be exposed, ask for a masked view or a generated sample preserving distributions, cardinalities and null rates. Insist on realistic skew — a uniform synthetic dataset hides every performance problem you were hired to find.

Working with the people

- [ ] **Find the person who actually knows how things work.** It is rarely the architect. It is usually someone who has been there seven years and maintains the critical job. Buy them coffee in week one.
- [ ] **Do not criticise the existing system in front of its author.** Every bad design was a reasonable decision under constraints you do not know about. Ask "what drove this?" instead.
- [ ] **Ship something small in week one.** A cost report, a freshness dashboard, one fixed query. Credibility compounds and it buys you latitude later.
- [ ] **Work in their tools.** Their git, their ticketing, their Slack, their naming conventions. A perfect system in your tooling is a system they will abandon.
- [ ] **Write down decisions.** Short ADRs: context, options, decision, consequences. Six months later this is what prevents the "why did we do it this way?" spiral.
- [ ] **Demo weekly to the business, not just to engineering.** The person funding this needs to see progress in their own terms.
- [ ] **Plan the handover from day one.** Pairing sessions, a recorded walkthrough, a runbook, and a named owner per pipeline — agreed in writing before you leave.

> ⚠ **The handover test**
> Before you leave, do this: have a member of their team, not you, respond to a deliberately broken pipeline using only your runbook, while you watch and say nothing. Every gap you see is a gap that will become a 2 a.m. phone call. Fix them, then repeat. This single exercise is worth more than any amount of documentation written in isolation.

Scoping a data engagement honestly

Scope	Realistic duration	Deliverable
Assessment and roadmap	2-3 weeks	Landscape map, cost audit, prioritised gap list, target architecture, estimate
One source to one mart, end to end	3-5 weeks	Ingestion, staging, dbt models, tests, orchestration, dashboard, runbook
Migrate N legacy jobs to ELT	1-2 weeks per non-trivial job after the first three	Parallel-run evidence, reconciliation tests, cutover plan
Real-time pipeline (CDC to lakehouse)	6-10 weeks	Connector, topics, sink, merge, monitoring, failover runbook
RAG document pipeline over an enterprise corpus	4-8 weeks	Incremental ingestion, chunking, ACL propagation, eval set, reindex job

> ℹ **The estimate that is always wrong**
> Whatever you estimate for ingestion, double it, because you do not yet know which of the four sources has an undocumented API, a 2 GB daily SFTP file in fixed-width EBCDIC, or an owner on three months' leave. Transformation estimates are usually accurate; ingestion estimates are usually fiction. Say this out loud when you give the estimate and you will be forgiven for it later.

Interview Questions

Walk me through what happens when you run `dbt build`.

dbt parses every `.sql` and `.yml` file, renders the Jinja to resolve `ref()`, `source()`, macros and variables, and builds a DAG from those dependencies, producing a `manifest.json`. It then executes nodes in topological order, respecting the thread count. For each model it wraps the compiled select

in DDL appropriate to the materialisation — `create or replace view`, `create or replace table as`, or for incremental models a `merge` or insert against the existing relation, creating it first if it does not exist. Crucially, `build` interleaves tests, snapshots and seeds with model execution in DAG order, so if a staging model's uniqueness test fails, the downstream models that depend on it are skipped rather than built on bad data. That is the difference from `dbt run && dbt test`, where bad data reaches your marts before any test runs. Results are written to `run_results.json`, which you can feed into monitoring or into a subsequent `--select state:modified+` run in CI.

A nightly pipeline failed at 02:00. You find out at 09:00 because an analyst complains. What do you fix?

Two things, in order. First the immediate data problem: identify the failed task and logical date, confirm the loads are idempotent, clear the failed task and its downstream, rerun, and verify row counts and a reconciliation query against the source. Second — and more important — fix the detection gap, because a seven-hour mean time to detection by human complaint is the actual defect. That means a failure callback from the orchestrator into the team's alert channel with the DAG, date and error, plus an independent freshness heartbeat that asserts `max(_loaded_at) > now() - SLA` for every tier-1 table and runs outside the orchestrator so it still fires when the orchestrator itself is down. Then I would write a short incident note and add whichever test would have caught the root cause earlier. The pattern I am applying is that every incident should produce exactly one new automated check.

Explain the difference between an incremental model with and without a lookback window.

Without a lookback, the incremental filter is typically `where event_ts > (select max(event_ts) from {{ this }})`. That is correct only if data always arrives in event-time order, which is never true in practice — mobile clients buffer offline, partner feeds are late, upstream systems retry, and some rows are backdated deliberately. Anything whose event time is older than the current maximum will never be picked up, so you lose data silently and permanently. A lookback changes the filter to `≥ max(event_ts) - interval 'N days'`, which reprocesses a window on every run. Combined with a `merge` on the unique key, the reprocessing is idempotent — the same rows are simply rewritten with the same values. You size N from measured lateness, typically p99.9, not from intuition: query the distribution of `datediff(hour, event_ts, _ingested_at)` over the last month. The trade-off is cost, since you reprocess N days of data every run, so for very large tables you combine a short lookback with a periodic full refresh or a separate late-data reconciliation job.

What is a slowly changing dimension type 2 and how do you query it correctly?

SCD2 preserves history by never updating a row in place. When a tracked attribute changes, you close the existing row by setting `valid_to` and `is_current = false`, and insert a new row with a new surrogate key, the new values, `valid_from` set to the change time, `valid_to` set to a high sentinel like 9999-12-31, and `is_current = true`. There are then two correct query patterns and they answer different questions. For "current state", filter on `is_current`. For "as it was at the time of an event" — which is what any historically accurate report needs — join with a half-open interval: `on d.customer_id = f.customer_id and f.event_ts ≥ d.valid_from and f.event_ts <`

`d.valid_to`. The half-open interval matters because closed intervals on both ends create either overlaps or gaps at the boundary, and you will get duplicate or missing rows. Using a sentinel end date rather than NULL keeps that predicate simple; with NULL you need an OR in every query and someone will forget it.

Your warehouse bill tripled in two months. How do you investigate?

I start with attribution before hypotheses. Query the metering history grouped by warehouse and by day to see whether it is a step change on a date or gradual growth, and which warehouse. Then join query history to find the most expensive queries by elapsed time and bytes scanned, grouped by user and by query shape. In my experience four causes cover most cases. First, a warehouse whose `auto_suspend` was removed or lengthened, so it bills while idle — compare hours billed against hours actually executing queries; utilisation under about 20% is the tell. Second, a BI tool refreshing on a short timer against daily data, keeping a warehouse permanently warm. Third, a model that was incremental and became full-refresh, often because someone added `--full-refresh` to the scheduled command and never removed it. Fourth, a query that lost partition pruning because someone wrapped the partition column in a function; look for queries where `partitions_scanned / partitions_total` is near 1 on a large table. I would fix the top two or three, set a resource monitor as a hard guardrail, and publish a weekly cost-by-warehouse view so it stays visible.

When would you choose streaming over batch, and what does it cost the customer?

I would choose streaming only when a business decision genuinely depends on data that is seconds or low minutes old — fraud blocking, operational control rooms, live personalisation, inventory that drives a customer-facing promise. For anything consumed by a human looking at a dashboard, micro-batch every 5-15 minutes is almost always sufficient and is perceived as real-time. The costs are mostly operational rather than infrastructural: a streaming job runs continuously, so it needs an on-call rotation, consumer lag monitoring, checkpoint and state management, schema compatibility enforcement at the registry, and it has no natural "just rerun it tomorrow" recovery path. Backfilling a streaming aggregate requires a separate batch job, so you often end up maintaining two implementations of the same logic. I would ask the customer what decision changes if the data is 15 minutes old instead of 15 seconds; if they cannot answer concretely, we build micro-batch and revisit in six months with evidence.

What is the difference between a data lake, a warehouse and a lakehouse, in terms a CTO would care about?

The difference a CTO should care about is who owns the storage and what it costs to change engines. A warehouse keeps your data in the vendor's storage behind their SQL engine, which gives excellent performance and governance but means switching vendors is a migration project. A lake keeps data as files in your own object storage in open formats, which is cheap and flexible but historically had no transactions, no schema enforcement and no indexes, so it degrades into an unqueryable swamp without discipline. A lakehouse adds a table format — Delta, Iceberg or Hudi — over those files, which restores ACID transactions, schema enforcement, time travel and statistics while keeping the data in storage you control and readable by multiple engines. Practically, the categories have converged:

Snowflake, BigQuery and Databricks all read Iceberg now. So my advice to a CTO is less "pick a category" and more "make sure your lowest layer is an open table format in your own account, so the engine above it is a reversible decision." That single choice preserves negotiating leverage for years.

How do you handle a schema change in a source system?

The handling depends on the type of change, and the goal is that breaking changes fail loudly and non-breaking ones flow through. Additive changes — a new nullable column — should be detected and either ignored or propagated depending on `on_schema_change` in dbt, which I set explicitly rather than relying on the default. Type widening, like int to bigint, is usually safe. Type narrowing, column removal and renames are breaking, and the right behaviour is to fail the staging model with a clear error rather than silently producing nulls — a rename in particular looks like a delete plus an add, so under backward compatibility the registry accepts it while every consumer silently starts getting defaults. Structurally, I defend against all of this in three places: a schema assertion at the landing boundary that compares the observed column set and types against an expected manifest; a schema registry with at least BACKWARD compatibility enforced for streaming sources; and a data contract with a notice period for breaking changes and a named owner. Then I make the staging layer the single place where a source quirk is absorbed, so one file changes rather than twelve.

Explain idempotency in a pipeline and why it matters so much.

A task is idempotent if running it twice with the same parameters leaves the system in the same state as running it once. It matters because retries, backfills and partial failures are normal operating conditions, not exceptions — if a task is not idempotent, every retry corrupts data and you cannot safely operate the pipeline at all. In practice it means three disciplines. Writes are delete-then-insert scoped to the partition the run owns, or a MERGE on a business key, never a bare append. Every date in the task body comes from the orchestrator's interval parameters rather than `current_date()`, so behaviour depends on the window being processed rather than on when the code happens to run. And external side effects — emails, webhooks, reverse-ETL pushes — are guarded by an idempotency key and suppressed during backfills, because the warehouse part being idempotent does not help if the last task re-sends ninety days of marketing emails. The test I apply is: can I kill this task at any instant and rerun it with no cleanup? If not, it is not production-ready.

A stakeholder says two dashboards show different revenue. How do you approach it?

I resist the urge to debug SQL immediately and instead establish what each number means, because in my experience the majority of these cases are definitional rather than defects. I ask for the exact filters, the date range, and the timezone of each dashboard, then trace both back through lineage to the models and ultimately to the source tables. Common causes, roughly in order: different timezone boundaries so one dashboard's "yesterday" is shifted by five and a half hours; different inclusion rules, like one counting cancelled orders or including internal test accounts; different handling of refunds, gross versus net; a fan-out from a fact-to-fact join inflating one side; and genuinely different freshness, where one source is a day behind. I would reproduce both numbers with a single query per definition, present the reconciliation line by line, and then — this is the durable fix — define the metric once in a governed model or semantic layer and point both dashboards at it, with a

reconciliation test against the source system so the drift cannot silently return. The output of the incident should be one canonical definition, not a fixed dashboard.

What are the risks of CDC with Debezium against a customer's production database?

The headline risk is the replication slot. Postgres cannot recycle WAL segments that a slot has not confirmed, so if your consumer stops — a deploy, a crash, a network partition, a Kafka outage — WAL accumulates and the source database's disk fills, which can take the customer's production system down. That is a severe incident caused by the analytics pipeline, and it is the fastest way to lose trust. Mitigations are monitoring the slot's byte lag with alerts well below free disk, setting `max_slot_wal_keep_size` so the database sacrifices the slot rather than itself, and having a documented re-snapshot procedure. Secondary risks: the initial snapshot can take long-held locks or saturate IO on a large table, so it should be run in a maintenance window or use an incremental snapshot mode; `REPLICA IDENTITY FULL` is needed for full before-images on updates and deletes but increases WAL volume; schema changes in the source propagate as events that can break consumers; and the DBA team must grant replication privileges and change `wal_level`, which requires a restart and therefore a change request. I would always raise the slot risk explicitly with the DBA team before proposing CDC, because they will respect that you knew about it, and because their buy-in is what makes the project possible.

You have two weeks at a customer and their data is a mess. What do you do?

I would pick one high-value, end-to-end slice rather than attempting breadth, because a working vertical slice changes the conversation and a half-finished platform does not. Week one: get access, map the landscape, and run the usage and cost audit queries, because that produces a concrete, surprising artefact I can present by day four. In parallel I agree with the sponsor on a single business question that matters — usually one dashboard or one metric — and its source tables. Week two: build that slice properly, which means raw landing preserved immutably, staging models with casts and renames, a dimension with SCD2 if history matters, a fact with a declared grain, tests on uniqueness, not-null, accepted values and a reconciliation test against the source, one orchestrated DAG that is idempotent and rerunnable, and a freshness check. Then a runbook and a handover session. I would deliberately leave the other eleven sources untouched and instead deliver a prioritised gap list with estimates. The outcome is one thing their team can trust and a pattern they can copy, which is far more valuable than eleven half-migrated pipelines and a slide deck.

Checklist and Lab Plan

Reading about pipelines does not make you able to build one under pressure in someone else's cloud account. Build the labs below in order; each one is deliberately small enough to finish and realistic enough to teach the failure modes. Then use the checklists on real engagements.

Lab 1 — Local lakehouse in a day

1. **Generate realistic data.**
   Write a Python script producing 10M order rows with skewed customers (80/20), 2% late-arriving events, 0.5% duplicates and 1% malformed rows. Realistic skew is the point — uniform data hides every problem.

2. **Land it raw.**
   Write NDJSON to `raw/orders/ingest_date=…/run_id=…/`. Never modify these files again.

3. **Convert to Parquet.**
   Use DuckDB or PyArrow. Measure file sizes for CSV, Snappy Parquet and Zstd Parquet. Measure query time for a filtered aggregate on each.

4. **Demonstrate the small-file problem.**
   Write the same data as 5,000 small files, time the same query, then compact to 256 MB files and time it again. Write the two numbers down; you will quote them to customers.

5. **Add a table format.**
   Convert to Delta or Iceberg (DuckDB, delta-rs or local Spark). Perform an upsert, a delete, time travel to the previous version, then roll back.

6. **Build dbt models.**
   Point dbt-duckdb at it: staging views, a dimension with a snapshot, an incremental fact. Add unique, not_null, relationships and one singular reconciliation test.

7. **Break it deliberately.**
   Add a duplicate key, a null in a required column, a new currency code. Watch which test fires and how clear the message is.

Lab 2 — Orchestration and reliability

1. Run Airflow or Dagster locally in Docker.

2. Build a daily DAG over Lab 1: extract window → validate → load (delete+insert by partition) → dbt build → freshness check.

3. Make every task take the logical date as a parameter. Prove idempotency by running the same date three times and asserting identical row counts.

4. Backfill 30 days with `max_active_runs=1`. Observe the duration and think about what that would cost on a real warehouse.
5. Inject failures: kill the worker mid-task, make the source return 500 for 30 seconds, feed a malformed file. Verify retries, quarantine and alerts behave as designed.
6. Write the runbook, then have someone else recover a failure using only your runbook.

Lab 3 — CDC and streaming

1. `docker compose` up Postgres, Kafka, Schema Registry, Kafka Connect with Debezium.
2. Configure logical replication, a publication, and a Debezium connector on one table.
3. Insert, update and delete rows; inspect the `before` / `after` envelopes on the topic.
4. Stop the consumer for ten minutes and watch `pg_replication_slots` lag grow. Set `max_slot_wal_keep_size` and watch the slot get invalidated instead.
5. Write a consumer that batches to Parquet and commits offsets only after a durable write. Kill it mid-batch and verify at-least-once with no loss.
6. MERGE the CDC output into a silver table. Verify deletes propagate and that reprocessing the same batch is a no-op.
7. Register an Avro schema, then try a breaking change and a non-breaking one. Observe the registry's response under each compatibility mode.

Lab 4 — The customer simulation

1. Take a public dataset you have not seen before and give yourself four hours.
2. Produce: a landscape map, a profiling report (row counts, null rates, cardinalities, date ranges, obvious quality issues), and five questions you would ask the owner.
3. Build one fact and one dimension with tests, orchestrated and documented.
4. Write a one-page summary for a non-technical sponsor: what you found, what you built, what you would do next, what it would cost.
5. That one page is the actual deliverable. Practise it until it takes twenty minutes.

Checklist: starting at a new customer

- [ ] Access request sent before day one, with justification per item and blocking items flagged.
- [ ] The five questions asked, answered, written down and confirmed back in writing.
- [ ] Landscape diagram drawn and corrected by their lead engineer.
- [ ] Usage, cost and freshness audit queries run; findings presented by end of week two.
- [ ] Data classification understood: what is PII, where may it be processed, who approves access.
- [ ] One small visible win shipped in week one.
- [ ] Scope agreed in writing as a vertical slice, not a platform.

Checklist: before a pipeline goes to production

- [ ] Raw data landed immutably with ingest timestamp, batch id and source path.
- [ ] Every task idempotent; reruns proven to produce identical results.
- [ ] Every date derived from the orchestrator's interval, never from wall clock.
- [ ] Lookback window sized from measured lateness, not guessed.
- [ ] Grain declared in a comment and enforced by a uniqueness test.
- [ ] Tests on uniqueness, not-null, accepted values, referential integrity and freshness.
- [ ] At least one reconciliation test against the source system for the headline metric.
- [ ] Quarantine path for invalid rows, with a count on a dashboard and a named triager.
- [ ] Retries configured with backoff; non-retryable errors fail fast.
- [ ] Alerting on failure **and** on absence, with the absence check outside the orchestrator.
- [ ] Compaction and vacuum/expire-snapshots scheduled as explicit tasks.
- [ ] Cost reviewed: warehouse size, auto-suspend, incremental vs full refresh, pruning verified in the plan.
- [ ] PII masked or tokenised; access granted to roles, not people; retention policy enforced.

- [ ] Runbook written, tested by someone else, and a named owner agreed.
- [ ] Lineage and documentation generated and published where their team will find it.

Checklist: handover

- [ ] Named owner per pipeline, agreed in writing by their manager, not just by them.
- [ ] Runbook validated by having their engineer recover a failure unaided.
- [ ] Recorded walkthrough of the architecture and the code, stored where they keep documents.
- [ ] All credentials rotated out of your control; your access revoked on an agreed date.
- [ ] Backlog of known gaps and deferred work, with your honest estimates attached.
- [ ] A 30-day check-in scheduled, and a clear statement of what is and is not included.